

PlayLink — Dokumentacja techniczna

Zespół PlayLink (IOAD 2025/2026)

2026-06-19

Spis treści

1 Playlink Documentation	3
1.1 Map	3
1.2 Quick Start	4
1.3 Conventions in these docs	4
2 Architecture Overview	4
2.1 System Topology	5
2.2 Technology Stack	5
2.3 Repository Layout	5
2.4 Core Domain Concepts	6
2.5 Key Flows	6
2.6 Security Model (summary)	7
2.7 Where to go next	7
3 Identity Authentication Flow	7
3.1 Sequence Diagram	7
3.2 Key Security Features	8
4 API Reference	9
4.1 Base URL	9
4.2 Authentication	9
4.3 Rate Limiting	9
4.4 Global Errors	9
4.5 Endpoint Overview	10
4.6 Health	10
4.7 Auth	11
4.8 Users	13
4.9 Rooms	14
4.10 Room Events / RSVP	20
4.11 Lobby Locations	24
4.12 Games	24
4.13 Admin	26
4.14 WebSocket Endpoints	27
4.15 Data Model	27
4.16 Configuration	27
5 Data Model Reference	28
5.1 Engine & Session Setup	28
5.2 Entity-Relationship Overview	28
5.3 RsvpStatus (StrEnum)	30
5.4 DEFAULT_LOBBY_LOCATION	30

5.5	Table: roommember — RoomMember	30
5.6	Table: room — Room	31
5.7	Table: message — Message	31
5.8	Table: roomevent — RoomEvent	32
5.9	Table: roomeventrsvp — RoomEventRsvp	32
5.10	Table: game — Game	33
5.11	Table: user — User	33
5.12	Table: nonce — Nonce	34
5.13	Cross-References	34
6	Realtime / WebSocket Protocol	35
6.1	Connection Managers	35
6.2	/ws/rooms — Global Room-List Channel	35
6.3	/ws/rooms/{room_name}/chat — Per-Room Chat Channel	37
6.4	Message Signing (EIP-191)	43
6.5	Cross-Links	44
7	Backend Configuration & Validation	44
7.1	Environment Variables	44
7.2	Admin Authorization	46
7.3	Rate Limiting	47
7.4	CORS	48
7.5	Username Validation	49
7.6	Related Documents	50
8	Migrations	50
8.1	Alembic Wiring	50
8.2	Operator Commands	51
8.3	Migration History	52
8.4	Per-Migration Detail	52
8.5	Current Schema	56
9	Frontend Routes	56
9.1	Route Map	56
9.2	BFF Authentication Model	57
9.3	/ — Splash	57
9.4	/about — About	57
9.5	/auth — Identity / Vault Keeper	57
9.6	/profile — Wanderer's Profile	58
9.7	/rooms — Game List	59
9.8	/rooms/[name] — Room Detail	61
9.9	/test — Developer Observatory	62
9.10	Cross-References	63
10	Frontend Component Library	63
10.1	Domain Components	63
10.2	Chrome UI Kit	65
11	Frontend Library — Client-Logic Reference	72
11.1	Public Environment Variables	72
11.2	auth.ts — Identity Derivation and Authentication	73
11.3	signing.ts — Chat Message Signing Contract	75
11.4	signingKey.ts — Session Key Custody	75
11.5	roomsStore.ts — Live Room List Store	76
11.6	chatStore.ts — Per-Room Chat WebSocket Store	78

11.7	<code>hintsContext.svelte.ts</code> — Svelte 5 Runes Hints System	81
11.8	<code>lobbyLocations.ts</code> — Lobby Location Data and Geo Helpers	83
11.9	<code>hooks.server.ts</code> — Session Hook	85
11.10	<code>app.d.ts</code> — Global Type Augmentation	85
11.11	Cross-References	86
12	Deployment & Operations	86
12.1	Local Development Quick Start	86
12.2	Docker Compose Breakdown	87
12.3	Container Startup	88
12.4	Docker-Aware DATABASE_URL	89
12.5	CI/CD Pipeline	89
12.6	Related Documentation	91
13	Testing Reference	91
13.1	Backend Tests	91
13.2	Frontend Tests	94
13.3	Related Documentation	96

1 Playlink Documentation

Playlink is a full-stack **LFG (Looking-For-Group)** web application for finding players for niche, retro, and less popular multiplayer titles. Users create short-lived game sessions (“rooms”) and browse open lobbies in real time. Identity is non-custodial: it is derived locally from a BIP39 mnemonic and proven to the backend with an ECDSA signature.

This is the developer documentation set. Start with the [architecture overview](#), then drill into the area you need.

1.1 Map

1.1.1 Overview

Document	Contents
Architecture	System topology, tech stack, domain concepts, core flows, security model.

1.1.2 Backend (FastAPI)

Document	Contents
API reference	Every REST endpoint: auth, request/response shapes, status codes, side effects.
Realtime reference	The /ws/rooms and /ws/rooms/{name}/chat WebSocket protocols and frame catalog.
Data model	SQLModel tables, fields, relationships, constraints, and the entity diagram.
Migrations	Alembic wiring, operator commands, and the ordered migration history.
Configuration	Environment variables, admin model, rate limiting, CORS, username validation.

1.1.3 Frontend (SvelteKit)

Document	Contents
Library & stores	Client logic: identity/signing, realtime stores, contexts, server hooks.
Routes	Pages, server load functions, form actions, and the BFF auth model.
Components	Domain components and the Diablo-II-themed chrome UI kit (prop tables).

1.1.4 Operations

Document	Contents
Testing	Backend pytest and frontend vitest suites, fixtures, and run commands.
Deployment	Docker Compose services, Dockerfiles, container startup, and CI/CD.

1.2 Quick Start

```
cp .env.example .env
cp frontend/.env.example frontend/.env
docker compose up --build
```

Services:

- Frontend — `http://localhost:3000`
- Backend (OpenAPI docs at `/docs`) — `http://localhost:8000`
- PostgreSQL — internal to the Docker network (no host port mapping by design)

See [deployment](#) for the full local and production setup, and the component READMEs (`backend/README.md`, `frontend/README.md`) for service-specific developer setup.

1.3 Conventions in these docs

- Each document opens with a one-paragraph purpose and a `> **Source:**` line naming the files it covers.
- Endpoints, environment variables, model fields, component props, and WebSocket frames are documented as tables sourced directly from the code.
- Cross-references between documents use relative links; follow them to avoid duplication.

2 Architecture Overview

Playlink is a full-stack **LFG (Looking-For-Group)** web application for organizing short-lived multiplayer game sessions (“rooms”) for niche, retro, and less popular titles. It pairs a single-module **FastAPI** backend with a **SvelteKit** Backend-for-Frontend (BFF) and a **PostgreSQL** database, orchestrated with Docker Compose. Identity is **non-custodial**: every user is a BIP39 mnemonic whose derived key proves ownership via an ECDSA (EIP-191) signature challenge.

Source: `backend/main.py`, `backend/models.py`, `frontend/src/`, `docker-compose.yml`, `backend/docs/auth-flow.md`

2.1 System Topology

flowchart LR

```

    Browser["Browser (Svelte client)<br/>ethers / viem signing"]
    SK["SvelteKit server (BFF)<br/>adapter-node :3000"]
    API["FastAPI :8000<br/>REST + WebSockets"]
    DB["PostgreSQL 18"]

    Browser -- "REST + WS (PUBLIC_BACKEND_URL / PUBLIC_WS_URL)" --> API
    Browser -- "form actions, httpOnly session cookie" --> SK
    SK -- "server-side fetch (BACKEND_INTERNAL_URL)" --> API
    API -- "SQLModel / SQLAlchemy" --> DB
  
```

The browser talks to the backend **directly** for most reads and for both WebSocket channels. It talks to the SvelteKit server only for the operations that must touch the httpOnly session cookie (login/logout and the authenticated form actions), which the SvelteKit server proxies to the backend with the JWT attached.

2.2 Technology Stack

Layer	Technology	Tooling
Frontend	SvelteKit 2 / Svelte 5 (runes), TypeScript	Bun
Signing	ethers + viem (BIP39 + EIP-191)	—
Backend	FastAPI (single module main.py), Python 3.14+	uv
ORM	SQLModel (SQLAlchemy + Pydantic)	—
Database	PostgreSQL 18 (prod) / SQLite in-memory (tests)	Docker
Migrations	Alembic	uv
Realtime	FastAPI WebSockets	—
Auth	BIP39 identity + ECDSA (eth_account), JWT HS256	—
Rate limiting	slowapi (REST) + per-address sliding window (chat WS)	—
Orchestration	Docker Compose (db, backend, frontend)	Docker

2.3 Repository Layout

```

.
├── backend/                FastAPI service
│   ├── main.py            All REST + WebSocket endpoints, managers, helpers, lifespan
│   ├── models.py         SQLAlchemy tables (User, Room, RoomMember, Message, RoomEvent, ...)
│   ├── database.py       Engine + session, Docker-aware DATABASE_URL rewrite
│   ├── usernames.py      Username format + profanity validation
│   ├── alembic/          Migration environment + versions
│   ├── tests/            Pytest suite (SQLite + TestClient)
│   └── docs/             auth-flow.md specification
├── frontend/             SvelteKit application
│   └── src/
│       ├── lib/          Client logic: auth, signing, stores, contexts, UI kit
│       ├── routes/       Pages + server loaders/actions
│       └── hooks.server.ts Session-cookie decode hook
├── docker-compose.yml    db + backend + frontend services
└── docs/                This documentation set
  
```

2.4 Core Domain Concepts

- **Identity / User** — derived from a 12-word BIP39 mnemonic; stored as an EIP-55 checksummed `identity_address` with a persistent random username. There is no password.
- **Room** — a named, capacity-limited, expiring game session tied to a `Game` and a `lobby_location`. The creator auto-joins; a user may own at most 3 active rooms.
- **RoomMember** — the link table joining users to rooms.
- **Message** — a chat message scoped to a room, optionally signed (EIP-191) by its sender for verifiable authorship.
- **RoomEvent / RoomEventRsvp** — an optional scheduled gathering on a room with per-member RSVPs (present / absent / maybe).
- **Game** — a catalog category, ordered by `sort_order`; admins can add/remove categories.
- **Nonce** — a one-time, hashed, short-lived challenge backing the auth flow.

A full field-level breakdown is in the [data model reference](#); schema history is in [migrations](#).

2.5 Key Flows

2.5.1 1. Authentication (non-custodial challenge-response)

```
sequenceDiagram
    autonumber
    participant B as Browser
    participant API as FastAPI
    participant SK as SvelteKit (BFF)
    B->>B: Derive identity from mnemonic (key never leaves client)
    B->>API: POST /auth/request-nonce { address }
    API->>API: Upsert user, store SHA-256(nonce), TTL 5m
    API-->>B: { nonce }
    B->>B: personal_sign("Sign in to Playlink\nNonce: <uuid>")
    B->>API: POST /auth/verify { address, nonce, signature }
    API->>API: Recover address, mark nonce used, issue JWT (HS256)
    API-->>B: { token, username }
    B->>SK: POST /auth?/login (token)
    SK-->>B: Set-Cookie session=<jwt> (httpOnly, sameSite=strict)
```

The JWT never reaches client-side JavaScript: the browser hands it to the SvelteKit login action, which stores it in an `httpOnly` cookie. Server loaders decode it (`jwt-decode`) and expose only the address/username/admin flag to pages. See [auth-flow.md](#) for the full specification and the [configuration reference](#) for JWT/nonce TTL tuning.

2.5.2 2. Room lifecycle and the live room list

Authenticated users create/join/leave rooms over REST. Every mutation broadcasts the full room list to all clients connected to the `/ws/rooms` WebSocket, and a background task prunes expired rooms every `ROOM_CLEANUP_INTERVAL_SECONDS`. See the [API reference](#) and [realtime reference](#).

2.5.3 3. Per-room chat with verifiable authorship

Each room exposes a JWT-authenticated, membership-gated chat channel at `/ws/rooms/{name}/chat`. Messages are signed with the sender's BIP39-derived key (EIP-191) so authorship is verifiable end-to-end rather than trusted on the JWT alone; the server reconstructs a canonical message string and rejects timestamps drifting more than `CHAT_SIGNATURE_SKEW_SECONDS` to blunt replay. The same channel carries event, RSVP, roster, room-closed, and kick frames. Full frame catalog in the [realtime reference](#).

2.6 Security Model (summary)

- **Non-custodial keys** — the private key never leaves the browser; the backend only verifies proofs.
- **Replay protection** — nonces are single-use, hashed at rest (SHA-256), short-lived, and invalidated on re-request.
- **BFF / XSS hardening** — the session JWT lives in an httpOnly, sameSite=strict cookie set by the SvelteKit server.
- **Authorization** — admin authority lives only in the ADMIN_ADDRESSES environment variable (no is_admin column); see [configuration](#).
- **Rate limiting** — slowapi caps REST traffic (default 10/minute); chat WS enforces a per-address sliding-window limit.
- **Network isolation** — Postgres has no host port mapping in Compose by design (Docker bypasses host firewalls); see [deployment](#).

2.7 Where to go next

- [Documentation index](#) — full map of these docs.
- [Backend API reference](#) · [Realtime reference](#) · [Data model](#)
- [Frontend library](#) · [Routes](#) · [Components](#)
- [Configuration](#) · [Migrations](#) · [Testing](#) · [Deployment](#)

3 Identity Authentication Flow

This document describes the challenge-response authentication mechanism used by Playlink.

3.1 Sequence Diagram

sequenceDiagram

```
autonumber
actor User
participant Browser as Client (Browser)
participant Backend as FastAPI (Auth Service)
participant Svelte as SvelteKit (BFF)
participant DB as PostgreSQL
```

Note over User,Browser: Phase 1: Request Challenge

User->>Browser: Enter 12-word mnemonic

Browser->>Browser: Derive Address (local only)

Browser->>Backend: POST /auth/request-nonce { "address": "0x..." }

Backend->>DB: Upsert User (identity_address, random username)

Backend->>Backend: Generate random UUID (nonce)

Backend->>Backend: Hash Nonce (SHA-256)

Backend->>DB: Invalidate old nonces for address

Backend->>DB: Store hashed nonce (TTL: 5m)

Backend-->>Browser: { "nonce": "uuid-..." }

Note over Browser: Phase 2: Local Signing

Note right of Browser: Message: "Sign in to Playlink\nNonce: <uuid>"

Browser->>Browser: Sign message with Private Key

Note over Browser,Backend: Phase 3: Verification

Browser->>Backend: POST /auth/verify { "address": "0x...", "nonce": "...", "signature": "0x"

```

Backend->>Backend: Hash provided nonce
Backend->>DB: Fetch matching unused/unexpired hashed nonce

alt Nonce found
  Backend->>Backend: Recover address from signature
  alt Signature valid
    Backend->>DB: Mark nonce as used
    Backend->>DB: Update user.last_login
    Backend->>Backend: Generate JWT (sub: address, username: string)
    Backend->>Browser: { "token": "jwt...", "username": "..." }
  else Signature Invalid
    Backend->>Browser: 401 Identity verification failed
  end
else Nonce Invalid/Expired
  Backend->>Browser: 401 Invalid or expired challenge
end

Note over Browser,Svelte: Phase 4: Secure Session
Browser->>Svelte: POST /auth?/login { "token": "jwt..." }
Svelte->>Browser: Set-Cookie session=jwt (httpOnly, Secure)
Svelte->>Browser: 200 OK / Success

```

3.2 Key Security Features

3.2.1 1. Replay Protection

- Nonces are **one-time use** (used flag in DB).
- Requesting a new nonce **invalidates** all previous ones for that address.
- Nonces have a short **TTL** (default: 5 minutes).

3.2.2 2. Database Integrity

- The database only stores the **SHA-256 hash** of the nonce.
- This prevents “pre-signing” attacks if the database is compromised.

3.2.3 3. Non-Custodial

- The private key **never leaves the client**.
- The backend only verifies the proof of ownership.

3.2.4 4. XSS Protection (BFF Pattern)

- The JWT is stored in an **httpOnly cookie** by the SvelteKit server.
- This prevents malicious browser scripts from stealing the session token.

3.2.5 5. Identity Normalization

- All addresses are converted to **EIP-55 checksum format** before storage or lookup.
- Users are assigned a random persistent username upon first registration.

4 API Reference

Complete reference for the Playlink REST API. The server is a Python FastAPI application served with uv. Responses are JSON unless noted otherwise.

The OpenAPI specification is available interactively at /docs when the server is running.

Source: backend/main.py, backend/models.py, backend/usernames.py

4.1 Base URL

Environment	URL
Local development	http://localhost:8000
Production	https://playlink.bartek.monster

4.2 Authentication

All authenticated endpoints require an HTTP Authorization header in the OAuth2 **Password Bearer** scheme:

Authorization: Bearer <jwt>

JWT tokens are obtained via the POST /auth/verify endpoint (see [Auth](#) below). The token is a standard HS256 JWT with a configurable expiry (default 60 minutes, see [configuration](#)). The dependency injection uses FastAPI's OAuth2PasswordBearer(tokenUrl="auth/verify", auto_error=False) — a missing or invalid token yields 401 with no redirect.

4.3 Rate Limiting

Every endpoint is guarded by `slowapi` with a configurable default rate limit (DEFAULT_RATE_LIMIT env var; default "10/minute"). Rate-limited requests receive:

Status	Body
429 Too Many Requests	{"detail": "Rate limit exceeded: 10 per 1 minute"} (or the configured limit)

See [configuration](#) for environment variable details.

4.4 Global Errors

Status	Condition
422 Unprocessable Entity	Request body fails Pydantic validation (missing fields, type errors, constraint violations). Returned automatically by FastAPI.
429 Too Many Requests	Rate limit exceeded.

4.5 Endpoint Overview

Method	Path	Auth	Description
GET	/	—	Health check
POST	/auth/request-nonce	—	Request a one-time authentication nonce
POST	/auth/verify	—	Verify signature and issue JWT
GET	/users/me	JWT	Get the authenticated user's profile
PATCH	/users/me	JWT	Update the authenticated user's username
GET	/rooms	—	List all active rooms
GET	/lobby-locations	—	List supported lobby locations
GET	/rooms/{room_name}	—	Get a single room's details
GET	/games	—	List game categories
POST	/rooms	JWT	Create a new room
POST	/rooms/{room_name}/join	JWT	Join a room
POST	/rooms/{room_name}/leave	JWT	Leave a room
POST	/rooms/{room_name}/kick/{member_address}	Admin	Kick a member from a room
GET	/rooms/{room_name}/event	—	Get the room's scheduled event
PUT	/rooms/{room_name}/event	JWT	Schedule or update the room's event
DELETE	/rooms/{room_name}/event	JWT	Cancel the room's event
PUT	/rooms/{room_name}/event/rsvp	JWT	Set the caller's RSVP for the event
POST	/games	Admin	Add a new game category
DELETE	/games/{name}	Admin	Delete a game category
DELETE	/rooms/{room_name}	Admin	Close a room

WebSocket endpoints are documented separately in [realtime.md](#): GET /ws/rooms (live room-list broadcast) and GET /ws/rooms/{room_name}/chat (per-room chat with EIP-191 signature verification).

4.6 Health

4.6.1 GET /

Basic health check. No authentication required.

Response 200 OK

```
{  
  "status": "ok",  
}
```

```
"service": "playlink-auth"
}
```

4.7 Auth

4.7.1 POST /auth/request-nonce

Request a one-time challenge nonce for an Ethereum identity address. The nonce is hashed (SHA-256) before storage; the plaintext is returned once and must be signed with EIP-191 `personal_sign` for the subsequent verification step.

Previous unused nonces for the same address are automatically invalidated.

Query parameter

Field	Type	Constraints
address	string	Valid Ethereum address (hex, 0x-prefixed, 42 chars). Converted to EIP-55 checksum.

Response 200 OK

```
{
  "nonce": "550e8400-e29b-41d4-a716-446655440000"
}
```

Errors

Status	Condition
400 Bad Request	address is not a valid Ethereum address format. {"detail": "Invalid identity address format"}

4.7.2 POST /auth/verify

Verify an EIP-191 `personal_sign` signature over a nonce and issue a JWT. The signed message format is:

Sign in to Playlink\nNonce: {nonce}

On success, the nonce is marked as used, the user's `last_login` is updated, and a JWT is returned.

Request body (VerifyRequest)

Field	Type	Constraints
address	string	Valid Ethereum address (EIP-55 checksum-compatible).

Field	Type	Constraints
nonce	string	The plaintext nonce returned by POST /auth/request-nonce.
signature	string	Hex-encoded EIP-191 signature (0x-prefixed, 132 chars) produced by signing the message above with the private key corresponding to address.

Response 200 OK

```
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "username": "alice_42",
  "is_admin": false
}
```

Field	Type	Description
token	string	HS256 JWT. Expiry configurable via JWT_EXPIRATION_MINUTES (default 60 min). Claims: sub (checksum address), username, is_admin, iat, exp, iss ("playlink-auth").
username	string	The user's display name; auto-generated if not yet set (format: user_<8 hex chars>).
is_admin	boolean	Whether the address is in the ADMIN_ADDRESSES whitelist. A false client must not interpret JWT claims to bypass this check on admin endpoints.

Errors

Status	Condition
400 Bad Request	address is not a valid Ethereum address format. {"detail": "Invalid identity address format"}
401 Unauthorized	Nonce not found, already used, or expired. {"detail": "Invalid or expired challenge"}
401 Unauthorized	Signature format is invalid (recover fails). {"detail": "Invalid signature format"}

Status	Condition
401 Unauthorized	Recovered address does not match address. {"detail": "Identity verification failed"}

4.8 Users

4.8.1 GET /users/me

Return the authenticated user's profile. Requires JWT.

Response 200 OK

```
{
  "id": 1,
  "identity_address": "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18",
  "username": "alice_42",
  "created_at": "2026-06-18T12:00:00Z",
  "last_login": "2026-06-18T14:30:00Z",
  "is_admin": false
}
```

Field	Type	Description
id	integer	Internal user ID.
identity_address	string	Ethereum address (EIP-55 checksum).
username	string \ null	Display name, or null if not yet set.
created_at	string	ISO 8601 timestamp (Z-suffixed).
last_login	string \ null	ISO 8601 timestamp of most recent successful POST /auth/verify.
is_admin	boolean	Whether the address is in the ADMIN_ADDRESSES whitelist.

Errors

Status	Condition
401 Unauthorized	Missing or invalid JWT. {"detail": "Not authenticated"} / "Token expired" / "Invalid token"
404 Not Found	User record not found (should not happen after successful auth). {"detail": "User not found"}

4.8.2 PATCH /users/me

Update the authenticated user's username. Requires JWT.

Request body (UpdateUserRequest)

Field	Type	Constraints
username	string	3-20 characters. Allowed: letters a-zA-Z, digits 0-9, underscore _, hyphen -. Must not match any entry in the profanity stoplist (LDNOOBW English), checked as whole-string and per_-/- token.

Response 200 OK

Same shape as GET /users/me.

Errors

Status	Condition
400 Bad Request	Username fails format validation. {"detail": "Username must be 3-20 characters: letters, numbers, _ or -."}
400 Bad Request	Username contains profanity. {"detail": "Username contains inappropriate language."}
401 Unauthorized	Missing or invalid JWT.
404 Not Found	User record not found.
409 Conflict	Username already taken (by another user). {"detail": "Username already taken."}

4.9 Rooms

4.9.1 GET /rooms

List all active (non-expired) rooms. No authentication required.

Each room is serialised from the Room ORM model. The response is a JSON array of objects.

Response 200 OK

```
[
  {
    "name": "dm-arena",
    "game": "Quake III Arena",
    "lobby_location": "eu-central",
    "players_max": 8,
    "description": "Casual deathmatch",
    "communicator_link": "https://discord.gg/example",
    "requirements": "Microphone preferred",
```

```

    "created_by": "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18",
    "created_at": "2026-06-18T12:00:00Z",
    "expires_at": "2026-06-18T13:00:00Z",
    "members": []
  }
]

```

Note: This endpoint returns the raw ORM model. For a richer view with member_addresses, players_active, and the nested event object, use GET /rooms/{room_name}.

Field	Type	Description
name	string	Room name (unique).
game	string	Game category name.
lobby_location	string	Lobby location code (e.g. "eu-central").
players_max	integer	Maximum number of members.
description	string \ null	Free-text description (max 500 chars).
communicator_link	string \ null	Voice/text chat link (max 500 chars).
requirements	string \ null	Free-text requirements (max 1000 chars).
created_by	string	Identity address of the room creator.
created_at	string	ISO 8601 timestamp.
expires_at	string	ISO 8601 timestamp. Rooms expire 60 min after creation by default (extended automatically when an event is scheduled).
members	array	List of User objects (currently empty array; membership details available via GET /rooms/{room_name}).

4.9.2 GET /rooms/{room_name}

Get a single room's full details, including the member roster and scheduled event. No authentication required.

Path parameter

Field	Type	Description
room_name	string	Room name (unique identifier).

Response 200 OK

```

{
  "name": "dm-arena",

```

```

"game": "Quake III Arena",
"lobby_location": "eu-central",
"players_max": 8,
"players_active": 2,
"member_addresses": [
  "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18",
  "0x1234567890abcdef1234567890abcdef12345678"
],
"members": [
  {
    "address": "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18",
    "username": "alice_42",
    "is_admin": false
  }
],
"description": "Casual deathmatch",
"communicator_link": "https://discord.gg/example",
"requirements": "Microphone preferred",
"created_by": "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18",
"expires_at": "2026-06-18T13:00:00Z",
"event": null
}

```

Field	Type	Description
name	string	Room name.
game	string	Game category.
lobby_location	string	Lobby location code.
players_max	integer	Max members.
players_active	integer	Current member count.
member_addresses	array[string]	Identity addresses of all members.
members	array[object]	Detailed member roster. Each entry: { address, username, is_admin }.
description	string \ null	Room description.
communicator_link	string \ null	Voice/text link.
requirements	string \ null	Member requirements.
created_by	string	Creator's identity address.
expires_at	string	ISO 8601 expiration timestamp.
event	object \ null	Scheduled event, or null if none. Shape matches GET /rooms/{room_name}/event response.

Errors

Status	Condition
404 Not Found	Room name does not exist. {"detail": "Room not found"}

4.9.3 POST /rooms

Create a new room. The creator is automatically added as a member.

Side effects: Broadcasts the updated room list over the /ws/rooms WebSocket to all connected global listeners. If the game name is new, a Game record is created automatically (via `_ensure_game`).

Rate limiting: Non-admin users may create at most **3 active rooms**. The check counts rooms where `created_by` matches the caller's address.

Auth: JWT required.

Request body (CreateRoomRequest)

Field	Type	Constraints
name	string	Room name (unique). Must not already exist.
game	string	Game category name. 1-100 chars. If the game does not exist it is auto-created at the end of the sort order.
lobby_location	string	Must be one of the 10 supported location codes (see GET /lobby-locations).
players_max	integer	Maximum number of members (no min/max constraint enforced by API beyond integer type).
description	string \ null	Max 500 characters.
communicator_link	string \ null	Must be a valid URL (Pydantic <code>HttpUrl</code>) if provided. Stored as string.
requirements	string \ null	Max 1000 characters.

Response 201 Created

```
{
  "status": "created",
  "room": "dm-arena"
}
```

Errors

Status	Condition
400 Bad Request	Non-admin user has reached the 3-room limit. {"detail": "You can create a maximum of 3 rooms."}
400 Bad Request	Game name is empty after trimming. {"detail": "Game name is required"}
400 Bad Request	lobby_location is not a recognised code. {"detail": "Unsupported lobby location"}
401 Unauthorized	Missing or invalid JWT.
409 Conflict	Room name already taken. {"detail": "Room name already taken"}

Status	Condition
422 Unprocessable Entity	Pydantic validation failure (e.g. <code>players_max</code> not an integer, <code>communicator_link</code> not a valid URL, <code>description</code> >500 chars).

4.9.4 POST /rooms/{room_name}/join

Join a room as a member.

Side effects: Broadcasts the updated room list over `/ws/rooms`, and emits a `roster_update` WebSocket frame to the room's chat channel.

Auth: JWT required.

Path parameter

Field	Type	Description
<code>room_name</code>	string	Room name.

Response 200 OK

```
{
  "status": "joined",
  "room": "dm-arena"
}
```

Errors

Status	Condition
400 Bad Request	Caller is already a member. {"detail": "You are already in this room"}
400 Bad Request	Room is full (member count >= <code>players_max</code>). {"detail": "Room is full"}
401 Unauthorized	Missing or invalid JWT.
404 Not Found	Room does not exist. {"detail": "Room not found"}
404 Not Found	User record not found (should not happen after successful auth). {"detail": "User not found"}

4.9.5 POST /rooms/{room_name}/leave

Leave a room. The caller's RSVP for the room's event (if any) is automatically dropped.

Side effects: Broadcasts the updated room list over `/ws/rooms`. Emits a `roster_update` frame to the room's chat channel. If the leaver had an RSVP, also emits an `event_update` frame with the updated event state.

Auth: JWT required.

Path parameter

Field	Type	Description
room_name	string	Room name.

Response 200 OK

```
{
  "status": "left",
  "room": "dm-arena"
}
```

Errors

Status	Condition
400 Bad Request	Caller is not a member. {"detail": "You are not in this room"}
401 Unauthorized	Missing or invalid JWT.
404 Not Found	Room does not exist. {"detail": "Room not found"}
404 Not Found	User record not found. {"detail": "User not found"}

4.9.6 POST /rooms/{room_name}/members/{member_address}/kick

Remove a non-admin member from a room. Only administrators (addresses in the ADMIN_ADDRESSES whitelist) may kick.

Side effects: - If the kicked member was the room creator, ownership is transferred to the kicking admin (room.created_by updated). - The kicked member's RSVP (if any) is removed. - A system message is inserted into the room's chat: "{username} was removed from the room by an administrator." - Emits message, roster_update, event_update (if event exists), and member_kicked WebSocket frames to the room's chat channel. - Broadcasts updated room list over /ws/rooms. - Disconnects the kicked member's active chat WebSocket connections (close code 4409).

Auth: Admin (JWT + address in ADMIN_ADDRESSES).

Path parameters

Field	Type	Description
room_name	string	Room name.
member_address	string	Identity address of the member to kick. Case-insensitive lookup.

Response 200 OK

```
{
  "status": "kicked",
  "member_address": "0x1234567890abcdef1234567890abcdef12345678",
}
```

```

    "member_username": "bob_99",
    "created_by": "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18",
    "ownership_transferred": false
}

```

Field	Type	Description
status	string	"kicked"
member_address	string	Identity address of the removed member.
member_username	string	Username of the removed member.
created_by	string	Room creator after the operation (may have been transferred).
ownership_transferred	boolean	Whether room ownership was transferred to the admin as a result of the kick.

Errors

Status	Condition
401 Unauthorized	Missing or invalid JWT.
403 Forbidden	Caller is not an admin. {"detail": "Admin privileges required"}
403 Forbidden	Target member is an admin. {"detail": "Administrators cannot be kicked"}
404 Not Found	Room does not exist. {"detail": "Room not found"}
404 Not Found	Target user not found. {"detail": "User not found"}
409 Conflict	Target user is not a member of the room. {"detail": "User is not a room member"}

4.10 Room Events / RSVP

4.10.1 GET /rooms/{room_name}/event

Get the room's scheduled event, including the RSVP roster. This endpoint is public — anyone may browse the schedule, but only members may RSVP.

Path parameter

Field	Type	Description
room_name	string	Room name.

Response 200 OK

```

{
  "starts_at": "2026-06-20T18:00:00Z",

```

```

"ends_at": "2026-06-20T20:00:00Z",
"created_by": "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18",
"created_at": "2026-06-18T12:00:00Z",
"updated_at": "2026-06-18T12:00:00Z",
"rsvps": [
  {
    "address": "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18",
    "username": "alice_42",
    "status": "present",
    "updated_at": "2026-06-18T12:05:00Z"
  }
]
}

```

Field	Type	Description
starts_at	string	ISO 8601 start timestamp.
ends_at	string	ISO 8601 end timestamp.
created_by	string	Identity address of the room creator (who scheduled the event).
created_at	string	ISO 8601 creation timestamp.
updated_at	string	ISO 8601 last-update timestamp.
rsvps	array[object]	List of RSVPs. Each entry: { address, username, status ("pre-sent" "absent" "maybe"), updated_at }.

Errors

Status	Condition
404 Not Found	Room does not exist. {"detail": "Room not found"}
404 Not Found	Room has no event scheduled. {"detail": "No event scheduled"}

4.10.2 PUT /rooms/{room_name}/event

Create or replace the room's scheduled event. Only the room creator may schedule. The room's expires_at is automatically extended to ends_at + 30 minutes (grace period) if the new expiry would be later.

Side effects: Broadcasts an event_update WebSocket frame to the room's chat channel. If the room expiry changed, also broadcasts the updated room list over /ws/rooms.

If the event time window changes (compared to an existing event), all existing RSVPs are dropped — members must reconfirm.

Auth: JWT required. address must match room.created_by.

Path parameter

Field	Type	Description
room_name	string	Room name.

Request body (ScheduleEventRequest)

Field	Type	Constraints
starts_at	datetime	Must be in the future (UTC). Converted to UTC if timezone-naive.
ends_at	datetime	Must be after starts_at. Converted to UTC if timezone-naive.

Response 200 OK

Same shape as GET /rooms/{room_name}/event.

Errors

Status	Condition
400 Bad Request	starts_at is not in the future. {"detail": "starts_at must be in the future"}
400 Bad Request	ends_at is not after starts_at. {"detail": "ends_at must be after starts_at"}
401 Unauthorized	Missing or invalid JWT.
403 Forbidden	Caller is not the room creator. {"detail": "Only the room creator can schedule an event"}
404 Not Found	Room does not exist. {"detail": "Room not found"}

4.10.3 DELETE /rooms/{room_name}/event

Cancel a scheduled event and clear all RSVPs. Only the room creator may cancel.

Side effects: Broadcasts {"type": "event_update", "event": null} to the room's chat channel.

Auth: JWT required. address must match room.created_by.

Path parameter

Field	Type	Description
room_name	string	Room name.

Response 200 OK

```
{
  "status": "cancelled",
  "room": "dm-arena"
}
```

Errors

Status	Condition
401 Unauthorized	Missing or invalid JWT.
403 Forbidden	Caller is not the room creator. {"detail": "Only the room creator can cancel the event"}
404 Not Found	Room does not exist. {"detail": "Room not found"}
404 Not Found	Room has no event scheduled. {"detail": "No event scheduled"}

4.10.4 PUT /rooms/{room_name}/event/rsvp

Upsert the caller's RSVP for the room's scheduled event. Only current room members may RSVP. One RSVP per user per event (previous status is overwritten).

Side effects: Broadcasts an rsvp_update WebSocket frame to the room's chat channel.

Auth: JWT required. Caller must be a member of the room.

Path parameter

Field	Type	Description
room_name	string	Room name.

Request body (SetRsvpRequest)

Field	Type	Constraints
status	string	One of "present", "absent", "maybe" (values of RsvpStatus enum).

Response 200 OK

```
{
  "address": "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18",
  "username": "alice_42",
  "status": "present",
  "updated_at": "2026-06-18T12:05:00Z"
}
```

Errors

Status	Condition
401 Unauthorized	Missing or invalid JWT.
403 Forbidden	Caller is not a room member. {"detail": "Only room members can RSVP to the event"}
404 Not Found	Room does not exist. {"detail": "Room not found"}
404 Not Found	User record not found. {"detail": "User not found"}
404 Not Found	Room has no event scheduled. {"detail": "No event scheduled"}

4.11 Lobby Locations

4.11.1 GET /lobby-locations

Return the list of supported lobby locations with display labels and approximate geo-coordinates, plus the default location code.

Response 200 OK

```
{
  "default": "eu-central",
  "locations": [
    { "code": "na-east", "label": "North America East", "lat": 39.0, "lon": -77.0 },
    { "code": "na-west", "label": "North America West", "lat": 37.8, "lon": -122.4 },
    { "code": "eu-west", "label": "Europe West", "lat": 51.5, "lon": -0.1 },
    { "code": "eu-central", "label": "Europe Central", "lat": 50.1, "lon": 8.7 },
    { "code": "eu-north", "label": "Europe North", "lat": 59.3, "lon": 18.1 },
    { "code": "sa-east", "label": "South America East", "lat": -23.6, "lon": -46.6 },
    { "code": "asia-east", "label": "Asia East", "lat": 35.7, "lon": 139.7 },
    { "code": "asia-south", "label": "Asia South", "lat": 1.3, "lon": 103.8 },
    { "code": "oceania", "label": "Oceania", "lat": -33.9, "lon": 151.2 },
    { "code": "africa-south", "label": "Africa South", "lat": -26.2, "lon": 28.0 }
  ]
}
```

Field	Type	Description
default	string	The default lobby location code ("eu-central").
locations	array[object]	List of location objects. Each: { code, label, lat, lon }.

4.12 Games

4.12.1 GET /games

List all game categories ordered by sort_order.

Response 200 OK

```
[
  "Quake III Arena",
  "Diablo II",
  "StarCraft",
  "Half-Life",
  "Unreal Tournament"
]
```

The five default games are seeded on first startup. Additional games may be added and removed via admin endpoints.

4.12.2 POST /games

Add a new game category at the end of the sort order.

Auth: Admin (JWT + address in ADMIN_ADDRESSES).

Request body (CreateGameRequest)

Field	Type	Constraints
name	string	Game name. 1-100 characters. Trimmed server-side.

Response 201 Created

```
{
  "name": "Team Fortress 2",
  "sort_order": 6
}
```

Errors

Status	Condition
400 Bad Request	Name is empty after trimming. {"detail": "Game name is required"}
401 Unauthorized	Missing or invalid JWT.
403 Forbidden	Caller is not an admin. {"detail": "Admin privileges required"}
409 Conflict	Game name already exists. {"detail": "Game already exists"}

4.12.3 DELETE /games/{name}

Delete a game category. If rooms are currently playing this game, the request is refused with 409 unless the query parameter `force=true` is set, in which case those rooms are closed first (same cascade as `DELETE /rooms/{room_name}`).

Side effects (when `force=true`): Each affected room is purged (messages, event, RSVPs, members), a `room_closed` WebSocket frame is broadcast to each room's chat channel, and the updated room list is broadcast over `/ws/rooms`.

Auth: Admin (JWT + address in ADMIN_ADDRESSES).

Path parameter

Field	Type	Description
name	string	Game name.

Query parameter

Field	Type	Default	Description
force	boolean	false	If true, close all rooms playing this game before deleting it.

Response 200 OK

```
{
  "status": "deleted",
  "game": "Team Fortress 2",
  "rooms_closed": ["dm-arena", "ctf-base"]
}
```

Field	Type	Description
status	string	"deleted"
game	string	The deleted game name.
rooms_closed	array[string]	Names of rooms closed as part of the deletion (empty array when none).

Errors

Status	Condition
401 Unauthorized	Missing or invalid JWT.
403 Forbidden	Caller is not an admin. {"detail": "Admin privileges required"}
404 Not Found	Game name does not exist. {"detail": "Game not found"}
409 Conflict	Active rooms are playing this game and force is not true. {"detail": "There are N active rooms currently playing this game."}

4.13 Admin

4.13.1 DELETE /rooms/{room_name}

Close (delete) a room. Cascades deletion of the room's chat messages, scheduled event, and all RSVPs.

Side effects: - Broadcasts `{"type": "room_closed", "room": "{room_name}"}` to the room's chat channel so clients redirect out. - Broadcasts the updated room list over `/ws/rooms`.

Auth: Admin (JWT + address in `ADMIN_ADDRESSES`).

Path parameter

Field	Type	Description
room_name	string	Room name.

Response 200 OK

```
{
  "status": "closed",
  "room": "dm-arena"
}
```

Errors

Status	Condition
401 Unauthorized	Missing or invalid JWT.
403 Forbidden	Caller is not an admin. <code>{"detail": "Admin privileges required"}</code>
404 Not Found	Room does not exist. <code>{"detail": "Room not found"}</code>

4.14 WebSocket Endpoints

The two WebSocket endpoints are documented in [realtime.md](#):

Path	Description
<code>/ws/rooms</code>	Unauthenticated live room-list broadcast.
<code>/ws/rooms/{room_name}/chat</code>	JWT-authenticated, membership-gated per-room chat with EIP-191 message signing.

4.15 Data Model

See [data-model.md](#) for entity shapes: Room, RoomMember, Message, RoomEvent, RoomEventRsvp, User, Nonce, Game.

4.16 Configuration

See [configuration.md](#) for environment variables (`DATABASE_URL`, `JWT_SECRET`, `JWT_ALGORITHM`, `NONCE_EXPIRATION_MINUTES`, `JWT_EXPIRATION_MINUTES`, `DEFAULT_RATE_LIMIT`, `ADMIN_ADDRESSES`, `ROOM_CLEANUP_INTERVAL_SECONDS`, etc.) and rate-limit details.

5 Data Model Reference

Source: backend/models.py, backend/database.py, backend/username.py

This document describes every database table, field, constraint, relationship, and enum used in the Playlink backend. The project uses **SQLModel** (a thin wrapper that fuses SQLAlchemy ORM models with Pydantic v2 validation) managed through **Alembic** for schema migrations. The runtime engine/session setup lives in database.py; the table classes live in models.py.

5.1 Engine & Session Setup

The backend connects to PostgreSQL (SQLite in tests) via a single DATABASE_URL environment variable:

- DATABASE_URL **MUST** be set at process start; if missing, database.py raises RuntimeError("DATABASE_URL environment variable is not set").
- If the URL contains @db: (the Docker Compose hostname) **and** the file /.dockerenv does **not** exist, the engine replaces @db: with @localhost: — this lets developers on the host machine reuse the same config without manual edits.
- The engine is created with create_engine(DATABASE_URL) (no special pool config).

Two utility functions are exported:

Function	Signature	Behaviour
create_db_and_tables()	() -> None	Calls SQLModel.metadata.create_all(engine). Used in tests; in production the schema is managed by Alembic migrations via entrypoint.sh (alembic upgrade head).
get_session()	() -> Generator[Session]	Opens a Session(engine), yields it as a FastAPI dependency, and closes it when the request ends.

5.2 Entity-Relationship Overview

```
erDiagram
    Game {
        int id PK
        string name UK
        int sort_order
    }

    Room {
        int id PK
        string name UK
        string game
        string lobby_location
    }
```

```

    int players_max
    string description
    string communicator_link
    string requirements
    string created_by
    datetime created_at
    datetime expires_at
}

RoomMember {
    int room_id PK, FK
    int user_id PK, FK
    datetime joined_at
}

Message {
    int id PK
    int room_id FK
    string sender_address
    string content
    datetime created_at
    string signature
    string sent_at
}

RoomEvent {
    int id PK
    int room_id FK, UQ
    datetime starts_at
    datetime ends_at
    string created_by
    datetime created_at
    datetime updated_at
}

RoomEventRsvp {
    int id PK
    int event_id FK
    int user_id FK
    string status
    datetime updated_at
}

User {
    int id PK
    string identity_address UK
    string username UK
    datetime created_at
    datetime last_login
}

Nonce {
    int id PK
    string identity_address

```

```

    string value UK
    datetime expires_at
    bool used
    int user_id FK
}

Room ||--o{ Message : "has"
Room ||--o{ RoomEvent : "has"
Room ||--o{ RoomMember : "membership"
RoomMember }o--|| User : "belongs to"
RoomEvent ||--o{ RoomEventRsvp : "rsvp"
RoomEventRsvp }o--|| User : "respondent"
User ||--o{ Nonce : "auth"
User ||--o{ RoomMember : "joined"

```

Foreign-key graph (acyclic): nonce.user_id → user.id, roommember.room_id → room.id, roommember.user_id → user.id, message.room_id → room.id, roomevent.room_id → room.id (ON DELETE CASCADE, UNIQUE), roomeventrsvp.event_id → roomevent.id (ON DELETE CASCADE), roomeventrsvp.user_id → user.id (ON DELETE CASCADE).

5.3 RsvpStatus (StrEnum)

Defined at module level in models.py. Used as the status column type in RoomEventRsvp.

Member	String value
present	"present"
absent	"absent"
maybe	"maybe"

5.4 DEFAULT_LOBBY_LOCATION

DEFAULT_LOBBY_LOCATION = "eu-central"

Fallback value for Room.lobby_location when the client omits the field.

5.5 Table: roommember — RoomMember

Many-to-many link table between User and Room. The composite primary key means a user can hold at most one membership per room; re-joining after leaving inserts a new row.

Field	Type	Constraints / Field() options	Description
room_id	int	primary_key=True, foreign_key="room.id"	Composite PK: room this membership refers to
user_id	int	primary_key=True, foreign_key="user.id"	Composite PK: member user
joined_at	datetime	default_factory=lambda: datetime.now(UTC)	Timestamp when membership was created

Relationships: None declared directly (used via `link_model=RoomMember` on `Room.members` and `User.rooms`).

5.6 Table: room — Room

The core grouping entity. A room represents a game session that expires after a configurable TTL.

Field	Type	Constraints / Field() options	Description
id	int \ None	primary_key=True, default=None	Auto-increment primary key
name	str	index=True, unique=True	Human-readable room name (unique, used in URLs)
game	str	(none)	Name of the game being played (free text, validated at API layer)
lobby_location	str	default=DEFAULT_LOBBY_LOCATION, index=True	Region/city of the lobby (e.g. "eu-central", "us-west")
players_max	int	(none)	Maximum number of members
description	str \ None	default=None, max_length=500	Free-text description of the session / expectations
communication_link	str \ None	default=None, max_length=500	Voice-chat invite link (Discord, TeamSpeak, etc.)
requirements	str \ None	default=None, max_length=1000	Hardware / software / skill requirements
created_by	str	index=True	Ethereum address (identity_address) of the room creator
created_at	datetime	default_factory=lambda: datetime.now(UTC)	Creation timestamp
expires_at	datetime	default_factory=lambda: datetime.now(UTC) + timedelta(minutes=60)	TTL; rooms past this timestamp are pruned by the query layer

Relationships:

Attribute	Type	Back-populates / link_model
members	list[User]	back_populates="rooms", link_model=RoomMember

5.7 Table: message — Message

Chat messages sent via WebSocket, stored for history replay. Supports EIP-191 signed messages (added in migration c4a9f1e7b6d2, Issue #59); legacy (pre-signing) messages have NULL signature and `sent_at`.

Field	Type	Constraints / Field() options	Description
id	int \ \nNone	primary_key=True, default=None	Auto-increment primary key
room_id	int	foreign_key="room.id", index=True	Owning room
sender_address	str	index=True	Ethereum address of the sender (identity_address)
content	str	(none)	Message body text
created_at	datetime	default_factory=lambda: datetime.now(UTC), index=True	Server-assigned timestamp when message was persisted
signature	str \ \nNone	default=None	EIP-191 hex signature over the canonical payload (Issue #59)
sent_at	str \ \nNone	default=None	Client-supplied ISO-8601 timestamp that was signed

5.8 Table: roomevent — RoomEvent

A single scheduled gathering attached to a room. Currently constrained to at most one event per room (room_id is UNIQUE); lifting that constraint later is a non-breaking change (Issue #62).

Field	Type	Constraints / Field() options	Description
id	int \ \nNone	primary_key=True, default=None	Auto-increment primary key
room_id	int	sa_column=Column("room_id", Integer, ForeignKey("room.id", ondelete="CASCADE"), unique=True, index=True, nullable=False)	Owning room (UNIQUE — one event per room)
starts_at	datetime	index=True	Scheduled start time
ends_at	datetime	index=True	Scheduled end time
created_by	str	index=True	Ethereum address of the room creator
created_at	datetime	default_factory=lambda: datetime.now(UTC)	Creation timestamp
updated_at	datetime	default_factory=lambda: datetime.now(UTC)	Last-update timestamp (bumped on reschedule)

5.9 Table: roomeventrsvp — RoomEventRsvp

A member's attendance declaration for a RoomEvent. Uses **upsert semantics** — an RSVP is created or updated in place. The unique constraint on (event_id, user_id) enforces one RSVP per user per event.

Field	Type	Constraints / Field() options	Description
id	int \ \nNone	primary_key=True, default=None	Auto-increment primary key

Field	Type	Constraints / Field() options	Description
event_id	int	sa_column=Column("event_id", Integer, ForeignKey("roomevent.id", ondelete="CASCADE"), index=True, nullable=False)	Target event (CASCADE-deleted with event)
user_id	int	sa_column=Column("user_id", Integer, ForeignKey("user.id", ondelete="CASCADE"), index=True, nullable=False)	Responding user (CASCADE-deleted with user)
status	RsvpStatus	(none)	One of present, absent, maybe
updated_at	datetime	default_factory=lambda: datetime.now(UTC)	Last modification timestamp

__table_args__:

__table_args__ = (UniqueConstraint("event_id", "user_id"),)

5.10 Table: game — Game

A curated list of games users can select when creating a room. Seeded at migration time with five entries: *Quake III Arena*, *Diablo II*, *StarCraft*, *Half-Life*, *Unreal Tournament*.

Field	Type	Constraints / Field() options	Description
id	int \ None	primary_key=True, default=None	Auto-increment primary key
name	str	index=True, unique=True	Display name of the game
sort_order	int	index=True	Ordinal for UI ordering (lower = earlier)

5.11 Table: user — User

Represents an authenticated participant. A user is created lazily on first successful wallet signature verification.

Field	Type	Constraints / Field() options	Description
id	int \ None	primary_key=True, default=None	Auto-increment primary key
identity_address	str	index=True, unique=True	Ethereum address (checksummed, from ECDSA recovery)
username	str	default_factory=lambda: f"user_{secrets.token_hex(4)}", index=True, unique=True	Auto-generated display name (e.g. user_a1b2c3d4); owner may change once
created_at	datetime	default_factory=lambda: datetime.now(UTC)	Registration timestamp

Field	Type	Constraints / Field() options	Description
last_login	datetime \\ None	default=None	Timestamp of most recent successful authentication

Relationships:

Attribute	Type	Back-populates / link_model
nonces	list[Nonce]	back_populates="user"
rooms	list[Room]	back_populates="members", link_model=RoomMember

5.12 Table: nonce — Nonce

Ephemeral authentication tokens used in the EIP-191 / ECDSA challenge-response flow. The value field stores a SHA-256 hash of the original nonce string (computed by the hash_nonce helper in main.py), preventing the raw nonce from persisting in the database.

Field	Type	Constraints / Field() options	Description
id	int \\ None	primary_key=True, default=None	Auto-increment primary key
identity_address	str	index=True	Ethereum address that requested the nonce
value	str	unique=True	SHA-256 hash of the challenge nonce
expires_at	datetime	(none — required)	Expiry timestamp; expired nonces are rejected by the API
used	bool	default=False	Single-use flag — set to True after successful verification
user_id	int \\ None	default=None, foreign_key="user.id"	FK to the user, populated only after successful auth

Relationships:

Attribute	Type	Back-populates / link_model
user	User \\ None	back_populates="nonces"

5.13 Cross-References

- **Migrations** — Alembic migration chain, all seven versions from initial tables to message signing columns.
- **API Reference** — REST endpoints that read and write each of these entities.
- **Realtime** — WebSocket frames that broadcast entity state changes (event-update, roster-update, rsvp-update).

6 Realtime / WebSocket Protocol

Playlink exposes two WebSocket endpoints for live push: a **global room-list channel** (unauthenticated) and a **per-room chat channel** (authenticated via JWT query parameter). Both are served from backend/main.py using two in-memory connection registries — ConnectionManager and RoomChatManager.

Source: backend/main.py (classes ConnectionManager, RoomChatManager; handlers websocket_rooms, websocket_chat; helpers get_rooms_payload, _members_payload, _serialize_event_state, _msg_dict, _chat_signing_message, _verify_chat_signature; constants CHAT_SIGNATURE_SKEW_SECONDS, SYSTEM_SENDER_ADDRESS, WS_LIMITS, MAX_MSGS, WINDOW), frontend/src/lib/roomsStore.ts, frontend/src/lib/chatStore.ts, frontend/src/lib/signing.ts.

6.1 Connection Managers

6.1.1 ConnectionManager — global room list

Member	Type	Description
active_connections	list[WebSocket]	Every connected /ws/rooms socket
connect(ws)	coroutine	Accept socket, append to list
disconnect(ws)	void	Remove from list (noop if absent)
broadcast(data: str)	coroutine	Send string to every connection; drop stale sockets that raise on send

Unreachable browsers (tab close, network loss) are pruned lazily during the next broadcast. There is no server-side heartbeat.

6.1.2 RoomChatManager — per-room chat

Member	Type	Description
rooms	dict[str, dict[WebSocket, str]]	room_name → { WebSocket → identity_address }
connect(room, ws, address)	coroutine	Accept socket, register under room key
disconnect(room, ws)	void	Unregister single socket; delete room key when empty
broadcast(room, payload: str)	coroutine	Send string to every connection in that room; prune stale sockets
disconnect_user(room, address, code=4409)	coroutine	Close every socket authenticated as address (case-insensitive)

6.2 /ws/rooms — Global Room-List Channel

GET /ws/rooms

6.2.1 Connection

- **No authentication.** The socket is accepted unconditionally and registered in the `ConnectionManager`.
- The server immediately sends a JSON payload — the current active rooms list — as the first frame.
- After that the socket sits in a `receive_text()` loop, consuming any inbound data silently (no actions taken on inbound messages).

6.2.2 Outbound Frames

The channel sends a single frame type: a **bare JSON array** of room-summary objects (no wrapping type field). Each array element:

```
{
  "name": "duel-arena",
  "game": "Quake III Arena",
  "lobby_location": "eu-central",
  "players_active": 2,
  "players_max": 4,
  "member_addresses": ["0xAbc...", "0xDef..."],
  "description": "Looking for a duel partner",
  "communicator_link": "https://discord.gg/abc123",
  "requirements": "Microphone required",
  "expires_at": "2026-06-19T12:00:00Z"
}
```

Field	Type	Description
name	string	Room slug (unique)
game	string	Game category name
lobby_location	string	Region code (e.g. eu-central, us-west)
players_active	number	Current member count (<code>len(r.members)</code>)
players_max	number	Capacity set at creation
member_addresses	string[]	Checksummed identity addresses of current members
description	string	nullable
communicator_link	string	nullable
requirements	string	nullable
expires_at	string	ISO 8601 with Z suffix

6.2.3 When Broadcasts Occur

The `ConnectionManager.broadcast(get_rooms_payload(session))` is called after every mutation that changes the visible room set:

Event	Endpoint / Source
Room created	POST /rooms
Room joined	POST /rooms/{name}/join
Room left	POST /rooms/{name}/leave
Member kicked	POST /rooms/{name}/members/{addr}/kick
Room closed (admin)	DELETE /rooms/{name}
Game deleted (admin)	DELETE /games/{name} (closes all rooms for that game)

Event	Endpoint / Source
Room expired (background)	cleanup_expired_rooms_task runs every 60s
Room expiry changed	Indirectly via event set/update when room expiry shifts

6.2.4 Expired-Room Pruning

Expired rooms (`Room.expires_at <= now`) are deleted synchronously inside `get_rooms_payload()` on **every** call, so the list broadcast never contains stale entries. The separate background task `cleanup_expired_rooms_task` (runs every 60s, interval set by `ROOM_CLEANUP_INTERVAL_SECONDS`) catches rooms that expire between `/ws/rooms` payload fetches.

Caution: The cleanup task does **not** send `room_closed` frames to the chat channel — it only refreshes the global room list. Chat connections to an expired room remain open until the client sends a message (at which point the DB lookup will fail) or the client disconnects.

6.3 /ws/rooms/{room_name}/chat — Per-Room Chat Channel

GET `/ws/rooms/{room_name}/chat?token=<JWT>`

6.3.1 Connection & Authentication

1. **JWT via query parameter.** The token query string carries the HS256 JWT obtained during login. Browsers cannot set custom WebSocket headers, so the token is passed in the URL.
2. The server decodes the JWT via `_decode_jwt()` to extract the identity address (sub claim). Failure → close code **4401** (Unauthorized).
3. The room must exist in the database. Not found → close code **4404** (Not Found).
4. The caller must be a current member of the room. Not a member → close code **4403** (Forbidden).

If all checks pass, the socket is registered with `RoomChatManager.connect()`.

6.3.2 Close Codes

Code	Meaning	When Sent
4401	Unauthorized	JWT missing, expired, or invalid
4403	Forbidden — not a member	Caller is not in the room's member list (checked on connect and per-iteration)
4404	Not Found	Room name does not exist
4409	Room membership terminated	Admin kick <code>disconnect_user</code> ; client reconnects and gets 4403
4429	Rate limited	Per-address message rate exceeded

On the frontend, close codes **4403** and **4409** set `kicked = true` and suppress reconnection. All other codes trigger exponential backoff reconnect.

6.3.3 Membership Re-validation

After every inbound message the server re-queries the RoomMember link table directly (not the ORM relationship cache):

```
membership = session.exec(
    select(RoomMember).where(
        RoomMember.room_id == room.id,
        RoomMember.user_id == user.id,
    )
).first()
```

If the member row is gone, the socket is closed with code **4403**. This ensures a kicked or removed member cannot continue writing even if their WebSocket stays open.

6.3.4 Per-Address Rate Limiting

A sliding window is enforced per identity address:

Variable	Env Variable	Default	Description
MAX_MSGS	WS_RATE_LIMIT_MAX_MESSAGES	10	Maximum messages per window per address
WINDOW	WS_RATE_LIMIT_TIME_WINDOW_SECONDS	10	Window duration in seconds

Rate limit state lives in the global `WS_LIMITS: dict[str, list[float]]` (in-memory, **not** shared across workers). On each inbound message the server prunes timestamps older than `WINDOW`, checks `count >= MAX_MSGS`, and if exceeded closes the socket with code **4429**.

6.3.5 Outbound Frame Types

Every outbound frame is a JSON object with a type string field and type-specific payload fields.

6.3.5.1 history Sent immediately after socket connect (before the read loop starts). Contains the last 50 messages in chronological order.

```
{
  "type": "history",
  "messages": [
    {
      "id": 1,
      "sender_address": "0xAbc...",
      "sender_username": "player1",
      "content": "gg",
      "created_at": "2026-06-18T20:00:00Z",
      "signature": "0x1234...",
      "sent_at": "2026-06-18T20:00:00.000Z",
      "verified": true
    }
  ]
}
```

Sub-field	Type	Description
messages	ChatMessage[]	Array of message objects (see ChatMessage table below)

6.3.5.2 message Broadcast to every connection in the room after a new message is stored.

```
{
  "type": "message",
  "message": {
    "id": 42,
    "sender_address": "0xAbc...",
    "sender_username": "player1",
    "content": "Hello!",
    "created_at": "2026-06-18T20:01:00Z",
    "signature": "0x1234...",
    "sent_at": "2026-06-18T20:01:00.000Z",
    "verified": true
  }
}
```

6.3.5.3 ChatMessage fields (_msg_dict)

Field	Type	Description
id	number	Auto-increment primary key
sender_address	string	Checksummed identity address
sender_username	string	Username of the sender (resolved at broadcast time)
content	string	Message body
created_at	string	ISO 8601 server timestamp (with Z)
signature	string	nullable
sent_at	string	nullable
verified	boolean	true when the server recovered a valid signer matching sender_address
kind	string	(optional, only when sender is system) "system"

A kind: "system" message is sent when the server injects an announcement (e.g. "player was removed from the room by an administrator"). The sender_address is "__system__".

6.3.5.4 event_update Sent when a room event is created, updated, cancelled, or its RSVP set changes.

```
{
  "type": "event_update",
  "event": {
    "starts_at": "2026-06-20T18:00:00Z",
    "ends_at": "2026-06-20T21:00:00Z",
    "created_by": "0xAbc...",
    "created_at": "2026-06-18T12:00:00Z",
    "updated_at": "2026-06-18T12:30:00Z",
    "rsvps": [
```

```

{
  "address": "0xAbc...",
  "username": "player1",
  "status": "present",
  "updated_at": "2026-06-18T12:30:00Z"
}
]
}
}

```

Sub-field	Type	Description
event	object	nullable

Event object fields:

Field	Type	Description
starts_at	string	ISO 8601 event start
ends_at	string	ISO 8601 event end
created_by	string	Checksummed address of the member who created the event
created_at	string	ISO 8601 creation timestamp
updated_at	string	ISO 8601 last-update timestamp
rsvps	RsvpEntry[]	Array of RSVP entries (one per member who responded)

RsvpEntry fields:

Field	Type	Description
address	string	Checksummed identity address
username	string	Username
status	string	One of "present", "absent", "maybe"
updated_at	string	ISO 8601 timestamp of this RSVP

6.3.5.5 rsvp_update Sent when a single member upserts their RSVP (via PUT /rooms/{name}/event/rsvp).

```

{
  "type": "rsvp_update",
  "rsvp": {
    "address": "0xDef...",
    "username": "player2",
    "status": "maybe",
    "updated_at": "2026-06-18T13:00:00Z"
  }
}

```

The frontend merges this into the local event state by replacing any existing RSVP from the same address (case-insensitive address dedup).

6.3.5.6 roster_update Sent when a member joins or leaves the room, or when a member is kicked.

```
{
  "type": "roster_update",
  "members": [
    { "address": "0xAbc...", "username": "player1", "is_admin": false },
    { "address": "0xDef...", "username": "player2", "is_admin": true }
  ]
}
```

Sub-field	Type	Description
members	RoomMember[]	Full current roster (not a diff); replaces client state

RoomMember fields:

Field	Type	Description
address	string	Checksummed identity address
username	string	Username
is_admin	boolean	Whether the address is in the ADMIN_ADDRESSES whitelist

6.3.5.7 room_closed Sent when an admin **deletes the room** or a **game is deleted** (which cascades to all its rooms).

```
{
  "type": "room_closed",
  "room": "duel-arena"
}
```

The frontend sets closed = true, tears down the socket, and suppresses reconnect.

6.3.5.8 member_kicked Sent when an admin removes a member via POST /rooms/{name}/members/{addr}/

```
{
  "type": "member_kicked",
  "member_address": "0xAbc...",
  "member_username": "player1",
  "created_by": "0xAdmin...",
  "ownership_transferred": true
}
```

Field	Type	Description
member_address	string	Checksummed address of the removed member
member_username	string	Username of removed member
created_by	string	Address of the new room owner (after potential transfer)
ownership_transferred	boolean	true when the removed member was the room owner

The frontend updates owner to created_by. If member_address matches the current session's address, it sets kicked = true and tears down the socket.

Immediately after this frame, the server calls disconnect_user() for the kicked address (close code 4409).

6.3.5.9 error Sent when a server-side validation rejects an inbound message (e.g. signature verification failed).

```
{
  "type": "error",
  "detail": "signature_invalid"
}
```

The frontend logs a warning; no state change occurs. The rejected message was **never stored** in the database.

6.3.6 Inbound Frame Types (Client → Server)

The client sends **one** type of frame — a chat message. The server expects a JSON object with at least a content field.

6.3.6.1 Unsigned (legacy / no-signer client)

```
{
  "content": "Hello, everyone!"
}
```

6.3.6.2 Signed (EIP-191)

```
{
  "content": "Hello, everyone!",
  "sent_at": "2026-06-18T20:01:00.000Z",
  "signature": "0x..."
}
```

Field	Type	Required	Description
content	string	Yes	Message body; trimmed, must be non-empty and ≤ 1000 chars
sent_at	string	With signature	ISO 8601 client timestamp included in the signing payload
signature	string	No	EIP-191 hex signature over the canonical message (see below)

Validation pipeline (server):

1. Parse JSON; reject malformed.
2. Trim content; reject empty or > 1000 chars.
3. Per-iteration membership check — close 4403 if no longer a member.
4. Rate-limit check — close 4429 if over limit.
5. If signature present:
 - Verify via _verify_chat_signature() (see [Message Signing](#) below).

- On failure: send {"type": "error", "detail": "signature_invalid"}, skip storage (do **not** close socket).
6. Create Message row, commit, then broadcast {"type": "message", "message": ...} to all room connections.
-

6.4 Message Signing (EIP-191)

Chat messages can be cryptographically signed with the sender's BIP39-derived ECDSA key so the server can prove authorship end-to-end rather than trusting the JWT alone. This is **opt-in**: clients without a signing key send unsigned messages (marked verified: false).

6.4.1 Canonical Message String

Both server and frontend construct this exact string:

```
PlayLink signed chat message
room={room_name}
sent_at={sent_at}
content={content}
```

- **Whitespace is significant.** The four lines are joined with literal \n (0x0A). No trailing newline.
- room is the raw room name (URL-encoded when connecting, but stored decoded).
- sent_at is the client-generated ISO 8601 string passed in the message payload.
- content is the trimmed message body.

Server implementation (_chat_signing_message):

```
def _chat_signing_message(room_name: str, content: str, sent_at: str) -> str:
    return (
        "PlayLink signed chat message\n"
        f"room={room_name}\n"
        f"sent_at={sent_at}\n"
        f"content={content}"
    )
```

Frontend equivalent (buildChatSigningMessage in signing.ts):

```
export function buildChatSigningMessage(room: string, content: string, sentAt: string): string {
    return `PlayLink signed chat message\nroom=${room}\nsent_at=${sentAt}\ncontent=${content}`;
}
```

See the [frontend library reference](#) for the ChatSigner interface and signChatMessage usage.

6.4.2 Signature Verification

The server verifies with _verify_chat_signature:

```
def _verify_chat_signature(
    room_name: str, content: str, sent_at: str, signature: str, address: str
) -> bool:
```

Step	Detail
Parse sent_at	Must be valid ISO 8601 via <code>datetime.fromisoformat()</code>
Timezone	Naive timestamps are treated as UTC
Clock skew	<code>abs(now - sent_at)</code> must be $\leq$ 120 seconds (<code>CHAT_SIGNATURE_SKEW_SECONDS</code>) — prevents replay
Recover signer	<code>encode_defunct(text=canonical_message)</code> then <code>Account.recover_message(message, signature)</code>
Compare	Recovered address lower-cased and compared to claimed address (lower-cased)

6.4.3 Replay Window

`CHAT_SIGNATURE_SKEW_SECONDS` = 120 (hardcoded, not configurable). The server compares `abs(now - sent_at)` against this value. Signatures outside the window are rejected as `signature_invalid`.

6.4.4 SYSTEM_SENDER_ADDRESS

System-generated messages (kick notifications, etc.) use the constant address `"__system__"` and always have kind: `"system"` in the payload. They are inserted by the server, not signed.

6.5 Cross-Links

- [REST API Reference](#) — REST endpoints for room CRUD, event management, RSVP, kick
- [Backend Configuration](#) — `WS_RATE_LIMIT_MAX_MESSAGES`, `WS_RATE_LIMIT_TIME_WINDOW_SECONDS`, `JWT_SECRET`, `ADMIN_ADDRESSES`
- [Frontend Library Reference](#) — `ChatSigner`, `buildChatSigningMessage`, `signChatMessage`, `ChatStore` interface
- [Data Model Reference](#) — `Room`, `Message`, `RoomEvent`, `RoomEventRsvp` SQLAlchemy schemas

7 Backend Configuration & Validation

This document covers every runtime configuration point of the Playlink backend: environment variables, the admin authorization model, rate limiting, CORS middleware, and username validation rules.

Source: `backend/main.py` (lines 56–104, 508–527, 77–80, 83–103, 309–320), `backend/usernames.py`, `docker-compose.yml`, `.env.example`

7.1 Environment Variables

The backend loads configuration from a `.env` file at the project root (`backend/../../.env`). On startup `main.py` calls `dotenv.load_dotenv()` if that file exists; the variables below are then read via `os.getenv()`.

Variable	Required	Default	Purpose
DATABASE_URL	Yes	—	SQLAlchemy database URL (e.g. postgresql+psycopg://user:pass@host:port). Raises RuntimeError at import time when unset.
JWT_SECRET	Yes	—	Symmetric key for HS256 JWT signing/verification. Raises RuntimeError at import time when unset.
JWT_ALGORITHM	No	HS256	JWT signing algorithm. Only HS256 is used.
NONCE_EXPIRATION_MINUTES	No	5	Lifetime of an auth nonce before it expires.
JWT_EXPIRATION_MINUTES	No	60	Lifetime of a JWT session token after issuance.
ROOM_CLEANUP_INTERVAL_SECONDS	No	60	Interval between background sweeps that delete expired rooms.
DEFAULT_RATE_LIMIT	No	10/minute	slowapi rate-limit string applied to every REST endpoint (see Rate Limiting).
WS_RATE_LIMIT_MAX_MESSAGES	No	10	Maximum number of chat messages a single address may send within the time window.
WS_RATE_LIMIT_TIME_WINDOW_SECONDS	No	10	Sliding time window (seconds) for the WebSocket chat rate limit.
ADMIN_ADDRESSES	No	"" (empty)	Comma-separated identity addresses granted admin privileges. Case-insensitive (see Admin Authorization).

7.1.1 Postgres Compose Variables

These are the individual variables in `.env.example` that feed into `DATABASE_URL`. The backend itself reads only `DATABASE_URL`; the compose variables are used by the db service in `docker-compose.yml`.

Variable	Example Value	Purpose
POSTGRES_USER	postgres	Database user name
POSTGRES_PASSWORD	postgres	Database password
POSTGRES_DB	playlink	Database name
POSTGRES_HOST	db	Database hostname (the Docker service name)
POSTGRES_PORT	5432	Database port

The assembled DATABASE_URL takes the form:

```
postgresql+psycpg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@${POSTGRES_HOST}:${POSTGRES_PORT}/${POSTGRES_DB}
```

7.1.1.1 @db → @localhost Rewrite When the backend runs outside Docker (e.g. during local development) and DATABASE_URL contains @db:, database.py rewrites it to @localhost: so the connection targets the locally-mapped Postgres port. The presence of /.dockerenv disables this rewrite.

7.2 Admin Authorization

Admin authority lives **entirely** in the ADMIN_ADDRESSES environment variable. There is no is_admin column on the User model — the only way to grant or revoke admin power is to edit .env and restart.

7.2.1 Address Parsing

main.py:83-98

```
def _parse_admin_addresses(raw: str | None) -> set[str]:
    """Parse the comma-separated `ADMIN_ADDRESSES` env value into a set.
    Addresses are lower-cased so membership checks are case-insensitive and
    independent of EIP-55 checksum casing. An empty/unset value yields an
    empty set — i.e. the system simply has no admins.
    """
    if not raw:
        return set()
    return {part.strip().lower() for part in raw.split(",") if part.strip()}
```

```
ADMIN_ADDRESSES: set[str] = _parse_admin_addresses(os.getenv("ADMIN_ADDRESSES"))
```

The resulting ADMIN_ADDRESSES set is a module-level mutable object so tests can hot-swap it.

7.2.2 Membership Check

main.py:101-103

```
def is_admin_address(address: str) -> bool:
    return address.lower() in ADMIN_ADDRESSES
```

Lower-casing both sides makes membership independent of EIP-55 checksum casing.

7.2.3 Auth Dependency

All admin-gated endpoints (room deletion, game CRUD, member kick) use the get_admin_address dependency:

```
# main.py:309-320
async def get_admin_address(
    address: Annotated[str, Depends(get_current_user_address)],
) -> str:
    if not is_admin_address(address):
        raise HTTPException(status_code=403, detail="Admin privileges required")
    return address
```

This function first resolves the caller's identity via normal JWT auth (`get_current_user_address`), then checks the whitelist. A forged `is_admin` claim in the JWT payload cannot grant access here because the source of truth is `ADMIN_ADDRESSES`.

7.2.4 Admin Flag in Responses

The JWT payload and the `GET /users/me` / `PATCH /users/me` responses include an `is_admin` boolean computed at response time:

```
# main.py:666-675
def _user_payload(user: User) -> dict:
    return {
        ...
        "is_admin": is_admin_address(user.identity_address),
    }
```

7.2.5 Enforcing Admin for Kick

The kick endpoint (`POST /rooms/{name}/members/{address}/kick`) receives the admin's resolved address as a dependency. The response payload marks members with `"is_admin": true` (see `_members_payload` at line 328).

7.3 Rate Limiting

Two independent rate-limiting mechanisms run in the backend.

7.3.1 REST API Rate Limiting (slowapi)

Configured at app startup:

```
# main.py:508-511
from slowapi import Limiter
from slowapi.util import get_remote_address
from slowapi.extension import _rate_limit_exceeded_handler

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)
```

- **Key:** Remote client IP address (via `get_remote_address`).
- **Default window:** 10/minute (configurable via `DEFAULT_RATE_LIMIT`).
- **429 handler:** The standard slowapi `_rate_limit_exceeded_handler`, which returns 429 Too Many Requests.
- **Scope:** Every REST endpoint carries `@limiter.limit(DEFAULT_RATE_LIMIT)`. This includes health check, auth, users, rooms, games, lobby-locations, events, and kick endpoints.

- **Test override:** The test `confest` sets `app.state.limiter.enabled = False` (see `confest.py`).

7.3.2 WebSocket Chat Rate Limiting

The `/ws/rooms/{name}/chat` endpoint enforces a **per-address** sliding-window limit independent of `slowapi`:

```
# main.py:77-80
WS_LIMITS: dict[str, list[float]] = defaultdict(list)

MAX_MSGS = int(os.getenv("WS_RATE_LIMIT_MAX_MESSAGES", "10"))
WINDOW = int(os.getenv("WS_RATE_LIMIT_TIME_WINDOW_SECONDS", "10"))
```

Inside the per-message receive loop (line 1486-1496):

1. Prune entries older than `WINDOW` seconds from the address's timestamp list.
2. If the remaining list length $\geq$ `MAX_MSGS`, close the WebSocket with code **4429**.
3. Otherwise append the current timestamp and process the message.

The `WS_LIMITS` dictionary is never pruned globally — stale entries for inactive addresses remain in memory until the process restarts, but they cost only a few floats per address.

7.4 CORS

CORS middleware is configured with an explicit whitelist of five origins:

```
# main.py:513-527
origins = [
    "https://playlink.bartek.monster",
    "http://localhost:3000",
    "http://127.0.0.1:3000",
    "http://localhost:5173",
    "http://127.0.0.1:5173",
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- **allow_credentials:** `True` — necessary for the session cookie used by the frontend BFF (see [API Reference](#)).
- **allow_methods:** `["*"]` — all HTTP methods permitted.
- **allow_headers:** `["*"]` — all HTTP headers permitted.
- No `expose_headers` is set; the default set applies.

The production origin (`https://playlink.bartek.monster`) points at the deployed frontend. The four localhost variants cover SvelteKit dev server (`:5173`) and the Node adapter production mode (`:3000`), with both localhost and `127.0.0.1` to handle DNS variations.

7.5 Username Validation

Defined in `backend/usernames.py` and called by `PATCH /users/me`.

7.5.1 Format Constraint

```
USERNAME_RE = re.compile(r"^[a-zA-Z0-9_-]{3,20}$")
```

Username must be: - 3–20 characters long - Alphanumeric (a–z, A–Z, 0–9), underscore (`_`), or hyphen (`-`) - No leading/trailing whitespace, no spaces

7.5.2 Profanity Stoplist

The backend vendors the [LDNOOBW English word list](#) at `backend/data/profanity_en.txt` (git blob `a438b9ca33af77341768bd6c63ce3e48e726b76e`, retrieved 2026-05-29). The check is fully offline.

```
# usernames.py:21-39
@lru_cache(maxsize=1)
def load_stoplist() -> frozenset[str]:
    """Load the profanity stoplist once, lowercased, blanks dropped."""
    lines = _STOPLIST_PATH.read_text(encoding="utf-8").splitlines()
    return frozenset(word.strip().lower() for word in lines if word.strip())

def contains_profanity(name: str) -> bool:
    """True if `name` as a whole, or any `_`/`-` token, is a stoplist word.

    Exact whole-string and per-token matching (rather than substring) avoids
    false positives like `assassin` or `classic` while still catching
    evasions such as `xX_fuck_Xx`.
    """
    stoplist = load_stoplist()
    lowered = name.lower()
    if lowered in stoplist:
        return True
    return any(token in stoplist for token in _TOKEN_SPLIT.split(lowered) if token)

• Whole-string match against the stoplist.
• Per-token split on _ and - (regex [_-]), with each non-empty token checked.
• No substring matching: a name like assassin passes because neither assassin nor any of its _-delimited tokens is in the stoplist as a complete entry.
```

7.5.3 Validation Function

```
# usernames.py:42-52
def validate_username(name: str) -> str | None:
    """Return an error code, or `None` when valid.
    Codes: "invalid_format" (fails the regex), "profane" (hits the stoplist).
    """
    if not USERNAME_RE.match(name):
        return "invalid_format"
    if contains_profanity(name):
        return "profane"
    return None
```

Return value	Meaning	HTTP status (in endpoint)
None	Valid	200 (success)
"invalid_format"	Regex failed	422 (see API Reference)
"profane"	Stoplist match	422

Duplicate usernames produce a 409 Conflict, detected via SQL IntegrityError on the user.username unique index.

7.6 Related Documents

- [API Reference](#) — endpoint signatures, request/response shapes, status codes.
- [Realtime](#) — WebSocket protocol, close codes including rate-limit code 4429.
- [Deployment](#) — Docker Compose setup, database configuration, CI/CD pipeline.

8 Migrations

This document describes the Alembic-based migration system used to manage the Playlink database schema. It covers wiring, common commands, the full migration history, and per-migration schema changes.

Source: backend/alembic/env.py, backend/alembic.ini, backend/alembic/versions/, backend/entrypoint.sh

8.1 Alembic Wiring

8.1.1 alembic/env.py

The environment script configures Alembic's offline and online migration modes:

- **Model import for autogenerate:** Every SQLAlchemy table class is imported so SQLAlchemy.metadata reflects the full schema:

```
from models import Game, Message, Nonce, Room, RoomEvent, RoomEventRsvp, User
```

`target_metadata = SQLAlchemy.metadata` tells Alembic which metadata to diff against the live database.
- **get_url() — local dev rewrite:** Reads DATABASE_URL from the environment. When running outside Docker (no /.dockerenv), the host @db: is rewritten to @localhost: — the same logic in backend/database.py. Raises RuntimeError if DATABASE_URL is unset.
- **load_dotenv:** Loads .env from the project root (backend/./.env) if the file exists.
- **Offline mode** (alembic upgrade --sql): Uses literal_binds=True and dialect_opts={"paramstyle": "named"} to produce portable SQL.
- **Online mode:** Builds a connection pool via engine_from_config with pool.NullPool (no persistent connection pooling across migration steps), then configures the context.

8.1.2 alembic.ini

Key	Value	Notes
script_location	alembic	Relative to backend/ where alembic.ini lives
prepend_sys_path	.	Enables from models import ... inside migrations
logger_root level	WARN	Suppresses engine SQL logging
logger_sqlalchemy level	WARN	
logger_alembic level	INFO	Shows migration progress
handler_console	StreamHandler, stderr	All log output

8.2 Operator Commands

All commands run from the backend/ directory with the project's managed Python environment (uv).

8.2.1 Create a revision (autogenerate)

```
uv run alembic revision --autogenerate -m "description of change"
```

Alembic diffs `SQLModel.metadata` against the live database and writes a new migration script under `alembic/versions/`. Review the generated output carefully — autogenerate can miss rename/type-change heuristics.

8.2.2 Apply pending migrations

```
uv run alembic upgrade head
```

Migrations are idempotent and transactional. Alembic tracks applied revisions in the `alembic_version` table (single row with `version_num`).

8.2.3 Check current revision

```
uv run alembic current
```

8.2.4 Roll back one step

```
uv run alembic downgrade -1
```

8.2.5 View history

```
uv run alembic history
```

8.2.6 Docker startup

In production, `entrypoint.sh` waits for the database (TCP port 5432 on host db, up to 30 attempts with 2-second sleeps) then runs:

```
alembic upgrade head
```

before starting uvicorn. See [deployment](#) for the full startup sequence.

8.3 Migration History

All migrations in dependency order (oldest first).

Revision ID	Parent (down_revision)	Summary
f273b29c941a	None (root)	Initial schema: game, room, user, nonce, roommember + seed games
a1b2c3d4e5f6	f273b29c941a	Add message table
17b0946aadca	a1b2c3d4e5f6	Add room metadata columns
4fb1ffbfc7d9	17b0946aadca	Add roomevent + roomeventrsvp
b7e2c1f4d8a3	4fb1ffbfc7d9	Add ends_at to roomevent + backfill
9c3d2e1f0a4b	b7e2c1f4d8a3	Add lobby_location to room
c4a9f1e7b6d2	9c3d2e1f0a4b	Add signature + sent_at to message

Current head: c4a9f1e7b6d2

8.4 Per-Migration Detail

8.4.1 f273b29c941a — initial

Root migration (down_revision: None). Creates five tables and seeds the game table.

Tables created:

Table	Columns	Constraints & Indexes
game	id (PK, int), name (str), sort_order (int)	ix_game_name (unique), ix_game_sort_order
room	id (PK, int), name (str), game (str), players_max (int), created_by (str), created_at (datetime), expires_at (datetime)	ix_room_name (unique), ix_room_created_by

Table	Columns	Constraints & Indexes
user	id (PK, int), identity_address (str), username (str), created_at (datetime), last_login (datetime, nullable)	ix_user_identity_address (unique), ix_user_username (unique)
nonce	id (PK, int), identity_address (str), value (str), expires_at (datetime), used (bool), user_id (int, nullable, FK → user.id)	ix_nonce_identity_address, unique on value
roommember	room_id (int, PK, FK → room.id), user_id (int, PK, FK → user.id), joined_at (datetime)	Composite primary key (room_id, user_id)

Seed data — 5 games inserted via `op.bulk_insert`:

name	sort_order
Quake III Arena	1
Diablo II	2
StarCraft	3
Half-Life	4
Unreal Tournament	5

8.4.2 a1b2c3d4e5f6 — add message

down_revision: f273b29c941a

Creates the message table for room chat.

message table:

Column	Type	Constraints
id	int (PK)	Auto-increment
room_id	int	FK → room.id, indexed (ix_message_room_id)
sender_address	str	Indexed (ix_message_sender_address)
content	str	
created_at	datetime	Indexed (ix_message_created_at)

8.4.3 17b0946aadca — add room metadata

down_revision: a1b2c3d4e5f6

Adds three nullable informational columns to the room table.

Column	Type	Nullable	Notes
description	VARCHAR(500)	yes	Free-text room description
communicator_link	VARCHAR(500)	yes	Voice/chat link (Discord etc.)
requirements	VARCHAR(1000)	yes	Prerequisites for joining

8.4.4 4fb1ffbf7d9 — add room event and RSVP

down_revision: 17b0946aadca

Creates the roomevent and roomeventrsvp tables for scheduling.

roomevent table:

Column	Type	Constraints
id	int (PK)	Auto-increment
room_id	int	FK → room.id (ON DELETE CASCADE), unique (one event per room — see Issue #62)
starts_at	datetime	Indexed (ix_roomevent_starts_at)
created_by	str	Indexed (ix_roomevent_created_by)
created_at	datetime	
updated_at	datetime	

roomeventrsvp table:

Column	Type	Constraints
id	int (PK)	Auto-increment
event_id	int	FK → roomevent.id (ON DELETE CASCADE), indexed
user_id	int	FK → user.id (ON DELETE CASCADE), indexed
status	ENUM 'rsvpstatus'	One of present, absent, maybe
updated_at	datetime	

Additional constraint: UniqueConstraint("event_id", "user_id") — one RSVP per user per event.

The downgrade explicitly drops the Postgres ENUM type via `sa.Enum(name="rsvpstatus").drop()` — a no-op on SQLite.

8.4.5 b7e2c1f4d8a3 — add event ends_at

down_revision: 4fb1ffbfc7d9

Adds an ends_at column to roomevent with a data backfill step.

Column	Type	Constraints
ends_at	datetime	NOT NULL, indexed (ix_roomevent_ends_at)

Backfill logic (for environments that already have rows from the prior migration):

1. Column is added as nullable.
2. For each existing row, ends_at is set to starts_at + 2 hours.
3. Column is altered to NOT NULL using batch_alter_table (SQLite compatibility — it rebuilds the table; Postgres uses a plain ALTER COLUMN).

New rows always provide ends_at via the API, so the backfill only affects pre-existing events.

8.4.6 9c3d2e1f0a4b — add room lobby_location

down_revision: b7e2c1f4d8a3

Adds a geographical lobby location column to room.

Column	Type	Default	Constraints
lobby_location	VARCHAR	'eu-central'	NOT NULL, server_default='eu-central', indexed (ix_room_lobby_location)

The column is added directly as NOT NULL with a server_default — no backfill needed since the default applies to existing rows on creation.

8.4.7 c4a9f1e7b6d2 — add message signature and sent_at

down_revision: 9c3d2e1f0a4b

Issue #59 — adds EIP-191 signature verification fields to message.

Column	Type	Nullable	Purpose
signature	str	yes	EIP-191 personal_sign hex string
sent_at	str	yes	Client-supplied ISO-8601 timestamp (e.g. "2026-06- 18T12:00:00Z")

Both columns are nullable so legacy messages remain valid and render as unverified. New signed messages populate both fields; unsigned messages leave them NULL.

8.5 Current Schema

See the [data model reference](#) for the full, up-to-date table definitions including all fields, constraints, and relationships at current head (c4a9f1e7b6d2).

9 Frontend Routes

Playlink uses SvelteKit’s server-side load functions and form actions to implement a Backend-for-Frontend (BFF) pattern. The JWT issued by the FastAPI backend is never exposed to client JavaScript; it is posted to the `?/login` form action via standard `fetch` and stored exclusively in an `httpOnly`, `sameSite=strict` session cookie with a 1-hour max age. Every server load function and action reads the cookie, decodes it with `jwtDecode`, and the page receives only derived data (address, username, `isAdmin`). Logout deletes the cookie. The `hooks.server.ts` handle also decodes the JWT on every request, populating `event.locals.user` with `{ address: string; isAdmin: boolean }` (see [library](#)).

Source: `frontend/src/routes/+layout.server.ts`, `frontend/src/routes/+layout.svelte`, `frontend/src/routes/+page.svelte`, `frontend/src/routes/about/+page.svelte`, `frontend/src/routes/auth/+page.server.ts`, `frontend/src/routes/auth/+page.svelte`, `frontend/src/routes/profile/+page.server.ts`, `frontend/src/routes/profile/+page.svelte`, `frontend/src/routes/rooms/+page.server.ts`, `frontend/src/routes/rooms/+page.svelte`, `frontend/src/routes/rooms/[name]/+page.server.ts`, `frontend/src/routes/rooms/[name]/+page.svelte`, `frontend/src/routes/test/+page.svelte`, `frontend/src/hooks.server.ts`

9.1 Route Map

Route	File(s)	Purpose	Auth Gating
<code>/</code>	<code>+page.svelte</code>	Splash screen — crest, title, “Enter Realm” CTA	None
<code>/about</code>	<code>about/+page.svelte</code>	Static informational page	None
<code>/auth</code>	<code>auth/+page.server.ts</code> , <code>auth/+page.svelte</code>	BIP39 mnemonic-based sign-in (3-phase auth), login/logout actions	None (renders differently when authenticated)
<code>/profile</code>	<code>profile/+page.server.ts</code> , <code>profile/+page.svelte</code>	View and edit username, display sigil/address/dates	Server redirects to <code>/auth</code> if no session
<code>/rooms</code>	<code>rooms/+page.server.ts</code> , <code>rooms/+page.svelte</code>	Real-time room list with WebSocket updates, filters, create/join/leave/admin actions	Load fetches games/locations publicly; actions require session
<code>/rooms/[name]</code>	<code>rooms/[name]/+page.server.ts</code> , <code>rooms/[name]/+page.svelte</code>	Room detail: chat via ChatStore, member list, event scheduling/RSVP, kick/close	Server requires session + membership (or admin)

/test	test/+page.svelte	Developer observatory — backend status, latency, raw JSON	None
-------	-------------------	--	------

9.2 BFF Authentication Model

The frontend never holds the JWT in JavaScript memory. The flow:

1. Client generates or receives a BIP39 mnemonic phrase.
2. `authenticate(mnemonic)` in `$lib/auth.ts` performs the three-phase handshake (see `/auth` below) against the backend.
3. On success, the backend returns a JWT (`{ token, username }`).
4. The client **does not store the token**; instead it POSTs the token as a form body field `token` to the `?/login` action.
5. The server action sets the cookie: `cookies.set('session', token, { httpOnly: true, sameSite: 'strict', secure: process.env.NODE_ENV === 'production', maxAge: 3600, path: '/' })`.
6. All subsequent server load functions and form actions read `cookies.get('session')` and decode it with `jwtDecode`.
7. The `hooks.server.ts` handle runs on every request, decoding the JWT into `event.locals.user` (`{ address, isAdmin }`). Expired or invalid cookies are deleted.
8. Logout calls `?/logout`, which runs `cookies.delete('session', { path: '/' })`.

The signing key (private key derived from the mnemonic) is kept in `sessionStorage` via `saveSigningKey()` so chat messages can be signed without re-entering the phrase. It is cleared on logout or tab close.

9.3 / — Splash

Purpose: Landing page for unauthenticated visitors.

Server data: None (no `+page.server.ts`). Page is fully static.

UI behavior: - Renders a centered splash with the D2 crest (Crest component, `size={104}`, `tone="gold"`), the “PLAYLINK” title, subtitle “Lobbies · Rooms · Kin”, and an `OrnateButton` linking to `/auth`. - Keyboard Enter navigates to `/auth`. - HintBar shows Enter: Enter Realm (gold tone).

9.4 /about — About

Purpose: Static informational page explaining the app purpose and usage ritual.

Server data: None (no `+page.server.ts`).

Content: - Identity model explanation (non-custodial, BIP39-based). - “The Ritual” three-step guide: 1. **Forge a Lobby** — pick a game, declare player count, name it. 2. **Watch the Hall** — open lobbies stream onto the game list in real time; filter by slots, status, ping. 3. **Stand in the Room** — join, chat, coordinate, leave or let the timer expire. - HintBar shows Esc: Back (red).

9.5 /auth — Identity / Vault Keeper

Server load (`auth/+page.server.ts`):

```
load({ cookies }): { user: { address: string; username: string } | null }
```

- Reads `cookies.get('session')`.
- If absent → returns `{ user: null }`.
- If present, `jwtDecode<SessionTokenClaims>(claims: sub, username, exp)`.
- If valid → returns `{ user: { address: decoded.sub, username: decoded.username } }`.
- If invalid/expired → deletes cookie, returns `{ user: null }`.

Form actions:

Action	Input	Backend call	Cookie	Returns
login	token (formData string)	None (cookie-only BFF)	Sets session cookie (httpOnly, sameSite=strict, maxAge 3600, secure in prod)	<code>{ success: true }</code>
logout	None	None	Deletes session cookie	<code>{ success: true }</code>

UI behavior:

Two states, controlled by `data.user`:

Unauthenticated (Vault Keeper): - 3-column MnemonicInput grid for a 12-word BIP39 phrase. - Keyboard shortcuts: G generates a new mnemonic (calls `generateMnemonic()`), C copies phrase to clipboard (fallback to `document.execCommand('copy')` when async clipboard API is blocked in HTTP context), Enter confirms. - `startAuth()` function: 1. Calls `authenticate(mnemonic)` from `$lib/auth.ts`: - **Phase 1 (challenge)**: POST `$PUBLIC_BACKEND_URL/auth/request-nonce?address=<address>`. Returns `{ nonce }`. Rate-limited (429 → "Rate limited"). - **Phase 2 (local signing)**: Derives `HDNodeWallet` from mnemonic. Saves signing key to `sessionStorage` via `saveSigningKey()`. Signs message "Sign in to Playlink\nNonce: `{nonce}`" with `identity.signMessage()`. - **Phase 3 (verification)**: POST `$PUBLIC_BACKEND_URL/auth/verify` with JSON body `{ address, nonce, signature }`. Returns `{ token, username }`. 2. POSTs the JWT to `*/login` with form data `{ token }`. 3. On success, calls `invalidateAll()` to re-run all active load functions. - Shows `SystemDialog` on error.

Authenticated (Active Covenant): - Displays the user's Sigil (88px), username, and address. - "Sever Covenant" button posts to `*/logout` (via hidden form). Keyboard L triggers it. - After logout, `invalidateAll()` re-runs loads so the page flips to the unauthenticated state.

9.6 /profile — Wanderer's Profile

Server load (`profile/+page.server.ts`):

```
load({ cookies }): { profile: { identity_address: string; username: string; created_at: string | null
```

- No session cookie → throw `redirect(303, '/auth')`.
- GET `$BACKEND_INTERNAL_URL/users/me` (or `$PUBLIC_BACKEND_URL` fallback) with `Authorization: Bearer <session>`.
- Non-OK response → throw `redirect(303, '/auth')`.
- Returns profile data: `{ identity_address, username, created_at, last_login }`.

Form actions:

Action	Input	Backend call	Returns
update	username (formData string, trimmed, 3-20 chars, [a-zA-Z0-9_-])	PATCH \$BAC-KEND_BASE/users/me with Authorization: Bearer <session> and JSON { username }	Success: { success: true, username: returned_username }. Failure: fail(status, { error, username? })

- Missing session → fail(401, { error: 'Not authenticated' }).
- Empty username → fail(400, { error: 'Username is required.' }).
- Server errors → fail(500, { error: 'Server error' }).

UI behavior: - Displays Sigil (88px), username, identity_address, created_at, last_login (formatted via toLocaleString(); – when null). - Editable username field with client-side validation: `^[a-zA-Z0-9_-]{3,20}$`. Submit button disabled when `!dirty || !clientValid || saving`. - Uses `use:enhance` for the `?/update` form. - `SystemDialog` shown for server errors. - `HintBar` shows Enter: Save Name (gold).

9.7 /rooms — Game List

Server load (rooms/+page.server.ts):

```
load({ cookies }): {
  isAuthenticated: boolean;
  user: { address: string } | null;
  isAdmin: boolean;
  games: string[];
  lobbyLocations: LobbyLocation[];
  defaultLobbyLocation: string;
}
```

- Reads session cookie and decodes it for address and isAdmin fields (silently fails if invalid).
- Fetches in parallel:
 - GET \$BACKEND_BASE/games → expects string[] of game names. On failure, games is empty [].
 - GET \$BACKEND_BASE/lobby-locations → expects { default?: string; locations?: { code: string; label: string; lat: number; lon: number }[] }. Parsed via `parseLobbyLocations()` with fallback to `FALLBACK_LOBBY_LOCATIONS/FALLBACK_LOBBY_LOCATION`.
- Never redirects — falls back to empty data so the page renders even when unauthenticated.

Form actions:

Action	Input	Backend call	Returns
create	name, game (or __custom__ + custom_game), lobby_location, players_max, optional description, communicator_link, requirements	POST \$BACKEND_BASE/rooms (JSON body)	{ success: true, message } or fail(status, { error })

Action	Input	Backend call	Returns
join	room_name	POST \$BAC-KEND_BASE/rooms/{name}	{ success: true, message } or fail(status, { error })
leave	room_name	POST \$BAC-KEND_BASE/rooms/{name}	{ success: true, message } or fail(status, { error })
deleteRoom	room_name	DELETE \$BAC-KEND_BASE/rooms/{name}	{ success: true, message } or fail(status, { error })
addGame	name (string)	POST \$BACKEND_BASE/games (JSON { name })	{ success: true, message } or fail(status, { error })
deleteGame	name (string), force (optional "true")	DELETE \$BAC-KEND_BASE/games/{name}	{ success: true, message = true } or fail(status, { error, conflict?, game? })

All actions require session; missing → fail(401).

create action details: - game can be "__custom__" — in that case customGame is used as the game name. - players_max is parsed via parseInt(..., 10). - Optional fields (description, communicator_link, requirements) are trimmed; empty strings become null.

deleteGame special handling: - 409 status (active rooms still playing the game) returns fail(409, { error, conflict: true, game }) so the UI can offer a force-delete confirmation.

UI behavior (1554-line page):

WebSocket room list: roomsStore singleton connects to WS \$PUBLIC_WS_URL/ws/rooms for live room list updates (see [realtime](#) and [library](#)). Rooms filter to active (unexpired) ones every reactivity tick.

Ping measurement: - Pings GET \$PUBLIC_BACKEND_URL/ every 15 seconds. - Estimated per-room latency combines API ping + Haversine distance between viewer's location and room's lobby location (divided by 100). - 3 pip buckets: LOW (≤80ms, 3 pips), MID (≤160ms, 2 pips), HIGH (>160ms, 1 pip).

Room timer display: - Live countdown per room showing MM:SS from expires_at. Low timer (<60s) triggers visual warning.

Filters: - Text search (searches room name, game, location label). - Slot count checkboxes (2, 4, 8, 16+). - Status: ANY / OPEN / FULL. - Ping: ANY / LOW / MID / HIGH.

Selection & side panel: - Click a room to select it; side column shows details (location, players, description, etc.). - Enter navigates to /rooms/{name} (requires membership or admin). Click outside deselects.

Create dialog: - N keyboard shortcut opens create dialog (requires auth). - Dropdown: game selector (from games array) or custom game name. - Lobby location: dropdown + "Detect" button using navigator.geolocation (nearestLobbyLocation() from \$lib/lobbyLocations). - Saved location persisted to localStorage key playlink.viewerLocation.

Admin features: - GameManager component for adding/deleting games (handles 409 conflict for active games). - Admin can delete any room directly from the list.

Toast notifications: Form action results (success/error) shown as toast, auto-dismissed after 4.5s.

Keyboard shortcuts: | Key | Action | |—|—| | / | Focus search input | | R | Reload page | | N | Open create dialog (authenticated) | | Enter | Open selected room (if member or admin) | | Esc | Close dialog / deselect / history.back() |

9.8 /rooms/[name] — Room Detail

Server load (rooms/[name]/+page.server.ts):

```
load({ params, cookies }): {
  roomName: string;
  roomGame: string;
  roomLocation: string;
  roomLocationLabel: string;
  description: string | null;
  communicatorLink: string | null;
  requirements: string | null;
  token: string; // raw JWT (for WebSocket auth)
  address: string;
  username: string;
  memberAddresses: string[];
  members: RoomMember[];
  event: RoomEventState | null;
  isAdmin: boolean;
  isMember: boolean;
  isPreview: boolean;
  createdBy: string;
  isCreator: boolean;
}
```

- No session → throw redirect(303, '/auth').
- Decodes session JWT for address, username, isAdmin. Invalid → /auth.
- GET \$BACKEND_BASE/rooms/{params.name} (public, no auth header). Failure/null → /rooms.
- If user is not a member AND not admin → redirect to /rooms.
- Returns room detail including members (typed RoomMember[]), event (typed RoomEventState | null), and the session token for WebSocket auth.
- isPreview = admin viewing a room they don't belong to (no WebSocket, read-only).

Form actions:

Action	Input	Backend call	Returns
closeRoom	None	DELETE \$BACKEND_BASE/rooms/{name}	{ success: true, closed: true } or fail(status, { error })
kickMember	member_address	POST \$BACKEND_BASE/rooms/{name}/members/{address}/kick	{ success: true, members: [] } or fail(status, { error })

Action	Input	Backend call	Returns
scheduleEvent	starts_at, ends_at (local datetime strings)	PUT \$BAC- KEND_BASE/rooms/{name} (JSON { starts_at: ISO, ends_at: ISO })	{ success: true, message } or fail(status, { error })
cancelEvent	None	DELETE \$BAC- KEND_BASE/rooms/{name}	{ success: true, message } or fail(status, { error })
setRsvp	status (one of present, absent, maybe)	PUT \$BAC- KEND_BASE/rooms/{name} (JSON { status })	{ success: true, message } or fail(status, { error })

All actions require session → fail(401).

scheduleEvent: starts_at/ends_at are local datetime-local input strings; converted to ISO UTC via `localInputToIsoUtc()`. Invalid dates → fail(400).

setRsvp: Validated against set { 'present', 'absent', 'maybe' }. Invalid → fail(400).

UI behavior (1181-line page):

WebSocket chat: - Members connect via `createChatStore(roomName, token, options)` from `$lib/chatStore` (see [library](#) and [realtime](#)). - ChatStore subscribes to messages, event, members, owner, closed, kicked stores. - Chat input: Enter sends, Shift+Enter newline. Auto-scrolls to bottom on new messages. - Chat messages are signed using the key from `loadSigner()` (from `sessionStorage`).

Member rail: - `partyMembers` computed reactive array with { address, username, isMe, isAdmin, isOwner }. - Each member shows a Sigil (small), username (or truncated address like 0x1234...5678), admin/owner badges.

Admin preview mode: - When `isPreview` is true (admin viewing a room they don't belong to): no WebSocket connection, no chat input, read-only view. State synchronized on form action invalidation (\$effect watching data).

RoomEvent component: - Render/edit/cancel events with `D2DatePicker` + `D2TimePicker` popover widgets. - RSVP buttons for members: Present / Absent / Maybe. - Live countdown to event start. Roster adjusts as members come/go.

Close/kick flow: - `closeRoom()`: POSTs `*/closeRoom`, then immediately navigates to `/rooms` via `goto()`. - `kickMember()`: POSTs `*/kickMember` with `member_address`. On success, removes member from local roster and cleans up RSVPs. If ownership transferred, updates `ownerAddress`. - When WebSocket announces `room_closed` or `member_kicked`: shows a modal `SystemDialog` (tone blood) with a 5-second countdown (`CLOSE_REDIRECT_SECONDS`), then auto-redirects to `/rooms`.

Dialogs: - `SystemDialog` modals for: kicked notification, room closed notification, kick target confirmation, close room confirmation.

HintBar: - Members: Enter: Transmit, ⬆Enter: Newline, Esc: Leave. - Admin preview: Esc: Back to Rooms.

9.9 /test — Developer Observatory

Purpose: Development-only backend health probe.

Server data: None (no `+page.server.ts`).

UI behavior: - On mount, fetches GET `$PUBLIC_BACKEND_URL/` and displays the raw JSON response. - Shows backend URL, HTTP status, latency (ms), and a PipMeter (3 pips `<80ms`, 2 `<250ms`, 1 otherwise, 0 on error). - R key re-pings the backend. - HintBar: R: Re-ping, Esc: Back. - Title: "PlayLink — Observatory".

9.10 Cross-References

- [Backend REST API Reference](#) — all endpoints called by form actions.
- [UI Components](#) — MnemonicInput, RoomEvent, GameManager, and all chrome components referenced above.
- [Frontend Library & Stores](#) — `auth.ts`, `roomsStore`, `chatStore`, `signingKey`, `hooks.server.ts`, `lobbyLocations`.
- [Realtime Protocol](#) — WebSocket frame types for rooms and chat.
- [Data Model](#) — Room, RoomEvent, RoomMember entity definitions.

10 Frontend Component Library

The Playlink frontend bundles a small set of domain-specific components (GameManager, MnemonicInput, RoomEvent) and an 18-piece chrome UI kit — a Diablo-II-themed widget library. All components use Svelte 5 runes conventions: `$props()` for prop declarations, `$bindable()` for two-way-bound state, `$state/$derived/$effect` for local reactivity, and `{@render children?.() }` for snippet/children slots.

Source: `frontend/src/lib/components/GameManager.svelte`, `MnemonicInput.svelte`, `RoomEvent.svelte`, `frontend/src/lib/components/chrome/*.svelte`

Shared design tokens and component styling live in `frontend/src/lib/styles/tokens.css` (CSS custom properties for colours, typography, spacing) and `frontend/src/lib/styles/chrome.css` (bevel/etched/small-caps atomic classes). `frontend/src/lib/global.css` provides the CSS reset, font declarations (Exocet display, Cinzel mono), and Diablo II cursor images. These files are imported by the root layout — individual components do not import them directly.

10.1 Domain Components

10.1.1 GameManager

Admin-only dialog for creating and deleting custom game categories. POSTs `*/addGame` and `*/deleteGame` form actions via `fetch` + `deserialize`.

Prop	Type	Default	Bindable	Description
<code>open</code>	<code>boolean</code>	—	yes	Dialog visibility
<code>games</code>	<code>string[]</code>	—	no	Current game category list
<code>onclose</code>	<code>() => void</code>	—	no	Called when dialog closes

The component wraps a `SystemDialog` with `tone="gold"` modal width="520px". Internal state tracks a new-game name input, a feedback message (type `'success' | 'error'`), a busy flag, and a conflict object.

Delete conflict flow: 1. `deleteGame(game)` sends `~/deleteGame` with `name`. 2. If the backend returns `{ type: 'failure', data: { conflict: true, error: "..." } }`, the conflict UI appears showing a “Force Delete” button and a “Cancel” link. 3. “Force Delete” calls `deleteGame(game, true)`, which appends `force: 'true'` to the form data. 4. On success, `invalidateAll()` triggers a re-fetch of the route data.

Keyboard: The name input accepts Enter key to trigger `addGame()` (via `onAddKey`).

See [routes](#) for the `~/addGame` and `~/deleteGame` action implementations in `rooms/+page.server.ts`.

10.1.2 MnemonicInput

A 12-word BIP39 mnemonic entry grid. Three columns of four slots, each showing a numbered input with per-word BIP39 dictionary validation via `ethers.wordlists.en.getWordIndex()`. Words are tracked as a `string[]` of 12 entries and synchronised with the bindable value (a space-joined string).

Prop	Type	Default	Bindable	Description
<code>value</code>	<code>string</code>	—	yes	Space-joined 12-word phrase

Bidirectional sync: - **Parent → child:** An `$effect` watches `value`. When it becomes a 12-word string, `words` is updated (only if the external value differs from the current internal state). When `value` clears, all slots reset. - **Child → parent:** An `$effect` joins words on space and sets `value`.

Navigation: - Space / Enter / Tab → advance to next slot. - Backspace on an empty slot → move to previous slot. - Paste → splits on whitespace, fills up to 12 slots, lowercase-strips non-alpha characters.

Validation per slot: - If non-empty and `wordlist.getWordIndex(word) === -1`, the slot gets class `invalid` (red tint). - If valid, class `valid`-filled (green tint) and a ▶ marker appear.

Used on the `/auth` route; see [routes](#).

10.1.3 RoomEvent

Scheduling and RSVP panel for a room event. Displays countdown, roster of RSVPs, and (for creator) edit/cancel controls; for members, RSVP buttons.

Prop	Type	Default	Bindable	Description
<code>event</code>	<code>RoomEventState</code> <code>\ null</code>		no	Current event data, or null if unscheduled
<code>isCreator</code>	<code>boolean</code>	—	no	Viewer owns the room
<code>isMember</code>	<code>boolean</code>	—	no	Viewer is a room member
<code>viewerAddress</code>	<code>string</code>	—	no	Viewer’s Ethereum address (lowercased internally)

Prop	Type	Default	Bindable	Description
members	{ address: string; username?: string }[]	—	no	All room members
formError	string \ \nnull	null	no	Server-side error from the parent form

States: 1. **No event, is creator** — “Schedule Event” button, toggles editing mode. 2. **No event, not creator** — “No event scheduled yet” message. 3. **Event scheduled, not editing** — Shows start/end time (formatted via Intl.DateTimeFormat), countdown timer, RSVP controls (if member), and roster. 4. **Editing** — D2DatePicker + D2TimePicker for start and end. On submit, validates local constraints then posts to `?/scheduleEvent`.

Countdown (updates every 1s via setInterval): - Before start: STARTS IN [d HH:MM:SS] - During: IN PROGRESS — [HH:MM:SS] LEFT - After end: ENDED

RSVP: Three buttons per member: Present ✓, Absent ✗, Maybe ?. Each POSTs `?/setRsvp` with status. The roster merges event.rsvps with members (addresses not in rsvps show “No Response”).

Editing form validation (validateDraft): - Start and end dates must be set. - End must be after start. - Start must be in the future.

Cross-links: uses D2DatePicker and D2TimePicker (see Chrome UI Kit below). RoomEventState, RsvpEntry, RsvpStatus types come from [chatStore](#). The form action `?/scheduleEvent / ?/cancelEvent / ?/setRsvp` is documented in [routes](#).

10.2 Chrome UI Kit

The chrome kit provides 18 Diablo-II-themed widgets. All components accept children?: `import('svelte').Snippet` where noted, and use `$props()` with TypeScript interface Props.

10.2.1 Cycler (generic <T>)

Cycles through an array of values with chevron buttons. Wraps around at boundaries.

Prop	Type	Default	Bindable	Description
values	T[]	—	no	Item pool
value	T	—	yes	Currently selected value
label	string \ \nundefined	—	no	Optional label text
format	(v: T) => string	—	no	Custom display formatter
disabled	boolean	false	no	Disables navigation
onchange	(v: T) => void	—	no	Called with the new value on change

Uses `$effect` to initialise value to `values[0]` if undefined. Left/right chevron buttons pulse on click for 120ms.

10.2.2 Crest

An SVG Diablo-II-style shield crest with Celtic knotwork, compass-star, and rivets.

Prop	Type	Default	Bindable	Description
size	number	96	no	Width/height in px
tone	'gold' \ \n'bone' \ \n'iron'	'gold'	no	Colour variant

Rendered as `<span aria-hidden="true" role="presentation">` with inline SVG.

10.2.3 D2DatePicker

Popover calendar date picker with a 42-cell (6×7) grid, month navigation, and auto-placement.

Prop	Type	Default	Bindable	Description
value	Date \ \nnull	—	no	Currently selected date
onChange	(d: Date) => void	—	no	Called when a date is picked (preserves existing time-of-day)
min	Date \ \nnull	null	no	Minimum selectable date
placeholder	string	'Pick a date'	no	Text when no date selected
ariaLabel	string	'Date'	no	Trigger button aria-label

Behaviour: - Weekday header starts on Monday. - Popover auto-flips above when space below is < 320px. - Clicking a non-current-month cell navigates to that month. - “Today” button in the header row (onclick: `jumpToday()`). - Closes on outside click and Escape. - Four bronze rivets in popover corners (matching InnerPanel aesthetic).

10.2.4 D2TimePicker

Popover time picker with two scrollable columns (hours 00–23, minutes at chosen granularity).

Prop	Type	Default	Bindable	Description
value	string	—	no	Current time as "HH:MM" (24h)
onChange	(value: string) => void	—	no	Called on pick with "HH:MM"
step	5 \ \n10 \ \n15 \ \n30	5	no	Minute picker granularity
ariaLabel	string	'Time'	no	Trigger button aria-label

Behaviour: - Parses value as HH:MM; falls back to 18:00 on parse failure. - Selected hour/minute scrolls into view on open. - Closes on outside click and Escape.

10.2.5 Hint

A keyboard-hint badge showing a key + label pair.

Prop	Type	Default	Bindable	Description
key	string	—	no	Key glyph (e.g. "E", "⌘", "M")
label	string	—	no	Description text
tone	'gold' \ 'green' \ 'amber' \ 'red' \ 'stone' \ 'blue'	'stone'	no	Colour accent
onclick	() => void	—	no	If set, renders as <button>; else as

The tone type (HintTone) is shared with [hintsContext](#). Most routes push HintEntry[] via provideHints() / getHintsState().

10.2.6 HintBar

Footer bar for keyboard hints. Renders a gold rule and a flex row of children. Hidden on touch devices via @media (hover: none) and (pointer: coarse).

Prop	Type	Default	Bindable	Description
children	Snippet	—	no	Hint elements (typically Hint components)

Rendered as <footer aria-label="Keyboard hints">.

10.2.7 InnerPanel

A stone-slab panel with bronze frame, four rivets, and CSS stone grain noise.

Prop	Type	Default	Bindable	Description
title	string \ undefined	—	no	Optional panel heading (etched gold, small-caps)
padded	boolean	true	no	Inner padding
children	Snippet	—	no	Panel body content
actions	Snippet	—	no	Actions rendered beside title in header

When title or actions is present, a header row with a gold rule is rendered above the body. The stone grain is a CSS pseudo-element with layered noise.

10.2.8 ListRow

A grid row with multiple states used for room/member lists.

Prop	Type	Default	Bindable	Description
selected	boolean	false	no	Selected state (shows gold pip)
header	boolean	false	no	Header row (no interaction, no pip)
disabled	boolean	false	no	Disabled state
member	boolean	false	no	Member state (shows dim pip)
onclick	() => void	—	no	Click handler; makes row interactive
children	Snippet	—	no	Row content

Interaction: If `!header && !disabled && typeof onclick === 'function'`, the row has `role="button"`, `tabindex="0"`, and responds to Enter/Space. Clicking fires a 180ms pulse animation. `aria-selected` and `aria-disabled` are set automatically.

10.2.9 OrnateButton

Themed button or link with multiple variants and sizes.

Prop	Type	Default	Bindable	Description
variant	'primary' \ 'secondary' \ 'danger' \ 'ghost'	'secondary'	no	Visual style
size	'sm' \ 'md' \ 'lg'	'md'	no	Button size
type	'button' \ 'submit' \ 'reset'	'button'	no	Button type (ignored when href set)
disabled	boolean	false	no	Disabled state
loading	boolean	false	no	Loading state (also disables)
href	string \ undefined	—	no	If set, renders <a> instead of <button>
fullWidth	boolean	false	no	Stretches to container width
onclick	(e: MouseEvent) => void	—	no	Click handler
children	Snippet	—	no	Button/link label

When href is set, renders as `<a role="button">` with `aria-disabled`. When loading, the inner span shows a pulsing slot animation (the `ornate__inner` div shifts in a CSS loop). `fullWidth` applies `width: 100%` via a class.

10.2.10 PageFrame

Top-level layout frame: stone wall backdrop with quatrefoil pattern, vignette overlay, four corner SVG shield ornaments, and gold edge hairlines.

Prop	Type	Default	Bindable	Description
children	Snippet	—	no	Page content

Rendered as `<div class="page-frame anim-frame-in">`. The frame-back layer adds a warm-dark-brown stone wall with CSS quatrefoil repeating pattern and a radial vignette darkening the edges. Four `<svg>` corner pieces in the frame-chrome layer use a gold-gradient stroke. Four `<span>` hairline edges (top/bottom/left/right) complete the frame. Content sits inside `.frame-inner`.

10.2.11 PipMeter

Pip indicator — a row of orbs showing a count with colour per tone.

Prop	Type	Default	Bindable	Description
value	number	—	no	Numeric value (floored to integer for lit pips)
total	number	3	no	Total pip count
tone	'good' \ \n'mid' \ \n'bad' \ \n'auto'	'auto'	no	Colour scheme
size	'sm' \ \n'md'	'md'	no	Pip size
glow	boolean	true	no	Outer glow on lit pips
label	string \ \nundefined	—	no	Text label below pips

Auto-tone (when tone = 'auto'): - lit $\geq 3 \rightarrow$ good (green) - lit = 2 $\rightarrow$ mid (amber) - lit = 1 $\rightarrow$ bad (red) - lit = 0 $\rightarrow$ good (default)

Lit pips animate with a scale-and-brightness pop keyframe (orbPop 300ms) when they transition from unlit to lit.

10.2.12 ProgressBar

Horizontal progress bar with optional tick marks and label.

Prop	Type	Default	Bindable	Description
value	number	—	no	Current value
max	number	100	no	Maximum value
height	number	6	no	Fill height in px
ticks	boolean	false	no	Show 25%/50%/75% tick marks
variant	'gold' \ \n'blood' \ \n'green'	'gold'	no	Fill colour

Prop	Type	Default	Bindable	Description
label	string \ \nundefined	—	no	Label text + numeric counter

The fill width is `clamped / safeMax * 100%` with a smooth transition: `width 0.35s ease`. `safeMax` ensures `max <= 0` becomes 1. The progressbar ARIA role and `aria-valuenow/aria-valuemax` are set.

10.2.13 SectionTitle

Section heading with a title, gold rule, and optional trail slot.

Prop	Type	Default	Bindable	Description
title	string	—	no	Heading text
size	'small' \ \n'normal'\n'large'	'normal'	no	Font size tier
tone	'gold' \ \n'bone' \ \n'blood'	'gold'	no	Title colour
children	Snippet	—	no	Content for the trail slot (right side of the rule)

Renders `<h3 class="title small-caps">`. The rule is a flex-growing hairline 1px background.

10.2.14 Sigil

Deterministic SVG sigil generated from an Ethereum address.

Prop	Type	Default	Bindable	Description
address	string	—	no	Ethereum address (0x-prefixed)
size	number	56	no	Width/height in px

Generation algorithm (parseSpec): 1. Lowercases the address, strips 0x prefix. 2. Requires ≥ 12 hex characters; pads deterministically to 16 bytes. 3. `bytes[0] % 6` $\rightarrow$ shape index (0–5). 4. `bytes[1] % 3` $\rightarrow$ symmetry type: 0 = vertical mirror, 1 = 4-fold rotation, 2 = 3-fold rotation. 5. `bytes[10] % 2 === 1` $\rightarrow$ useGold for accent colour. 6. `bytes[11] & 0x0f` $\rightarrow$ accent ring selector. 7. 25 cells (5×5 grid) derived from `bytes[2..]` via `(bytes[2 + i*4] >> (i % 8)) & 1`, then folded by the chosen symmetry.

Rendered as `<div class="sigil-frame bevel-out" aria-hidden="true">` with an inner SVG. Invalid addresses produce a fallback crossed-swords icon (X pattern).

10.2.15 StoneCheckbox

Diablo-II-themed checkbox with stone box and animated SVG checkmark.

Prop	Type	Default	Bindable	Description
checked	boolean	false	yes	Checked state
label	string \ undefined	—	no	Label text (small-caps)
disabled	boolean	false	no	Disabled state
size	'sm' \ 'md'	'md'	no	Checkbox size
onchange	(checked: boolean) => void	—	no	Called on toggle with new value

A real `<input type="checkbox" class="sr-only">` is used for accessibility; the visual box is a `<span>` with an SVG checkmark that scales in on checked. The label is a `<label>` wrapping the input, so clicking the visual box toggles the state.

10.2.16 SystemDialog

Modal, inline, or floating dialog with configurable tone, position, and footer slot.

Prop	Type	Default	Bindable	Description
open	boolean	false	yes	Dialog visibility
title	string	—	no	Dialog title
tone	'gold' \ 'blood' \ 'green' \ 'iron'	'iron'	no	Accent colour
modal	boolean	false	no	Dark overlay, Escape closes, body scroll locked
inline	boolean	false	no	Renders in-flow (no overlay)
position	'bottom- right' \ 'top- center' \ 'center'	'bottom-right'	no	Floating dialog position
width	string	'420px'	no	CSS width value
closeable	boolean	true	no	Permits closing; false hides close button and disables close
onclose	() => void	—	no	Called when dialog closes
children	Snippet	—	no	Dialog body content
footer	Snippet	—	no	Footer content (rendered below the body)

Four rendering modes: 1. **Modal** (modal: true) — dark overlay, centering, body overflow: hidden, Escape closes. 2. **Inline** (inline: true) — no overlay, rendered in the normal document flow. 3. **Floating** (default) — positioned absolutely per position value. 4. **Hidden** (open: false) — renders nothing.

Tone controls the heading rule colour, close-button colour, and overlay tint via CSS classes. The dialog fires onclose when closed, setting open = false.

10.2.17 Tab

Navigation tab for the top navigation bar. Uses \$app/state page store for active detection.

Prop	Type	Default	Bindable	Description
href	string	—	no	Route path
match	'exact' \ 'prefix'	'prefix'	no	URL matching strategy
label	string \ undefined	—	no	Tab label text
children	Snippet	—	no	Alternative to label for custom content

Active detection: When match = 'prefix', the tab is active when page.url.pathname === href or starts with href + '/'. When match = 'exact', only an exact match counts. Active tab shows a gold arrow indicator below the text. Uses data-sveltekit-preload-data="hover" for prefetching.

10.2.18 Tabs

Tab bar container — renders children in a grid-auto-flow: column row with top and bottom gold rules.

Prop	Type	Default	Bindable	Description
children	Snippet	—	no	Tab elements

At ≤ 600px viewport width, the row becomes overflow-x: auto with hidden scrollbar. The top and bottom rules are gradient fades from transparent → gold → transparent.

11 Frontend Library — Client-Logic Reference

This document describes the non-UI TypeScript/Svelte modules in frontend/src/lib/: authentication, chat-message signing, session key custody, WebSocket-backed stores (room list and per-room chat), a Svelte 5 runes hints context, and lobby-location helpers. The server-side hooks.server.ts and app.d.ts type augmentation are also covered here for completeness.

Source: frontend/src/lib/auth.ts, signing.ts, signingKey.ts, roomsStore.ts, chatStore.ts, hintsContext.svelte.ts, lobbyLocations.ts, frontend/src/hooks.server.ts, frontend/src/app.d.ts

11.1 Public Environment Variables

Variable	Used In	Purpose	Default
PUBLIC_BACKEND_URL	auth.ts	Base URL for HTTP REST calls	http://localhost:8000
PUBLIC_WS_URL	roomsStore.ts, chatStore.ts	Base URL for WebSocket connections	ws://localhost:8000

Both are accessed via `$env/dynamic/public (import { env } from '$env/dynamic/public')`.

11.2 auth.ts — Identity Derivation and Authentication

Three-phase authentication flow using BIP39 mnemonics and EIP-191 (`personal_sign`) signatures. Uses ethers v6.

11.2.1 getIdentityFromMnemonic(phrase: string): HDNodeWallet

Derives an ethers.HDNodeWallet (private/public key pair) from a 12-word BIP39 mnemonic phrase.

```
import { ethers, Mnemonic, HDNodeWallet } from 'ethers';

export function getIdentityFromMnemonic(phrase: string): HDNodeWallet {
  try {
    const mnemonic = Mnemonic.fromPhrase(phrase);
    return HDNodeWallet.fromMnemonic(mnemonic);
  } catch (e) {
    console.error('Wallet derivation failed:', e);
    throw new Error('Invalid mnemonic phrase');
  }
}
```

- **Input:** A space-delimited 12-word BIP39 phrase.
- **Output:** HDNodeWallet — wallet.address is the checksummed Ethereum address used as the user's identity.
- **Throws:** Error('Invalid mnemonic phrase') on parse/derivation failure.

11.2.2 generateMnemonic(): string

Generates a fresh 12-word BIP39 mnemonic from 16 bytes of entropy.

```
export function generateMnemonic(): string {
  return Mnemonic.fromEntropy(ethers.randomBytes(16)).phrase;
}
```

- **Return:** A space-separated phrase of 12 BIP39 words.

11.2.3 signMessage(identity: HDNodeWallet, message: string): Promise<string>

Signs an arbitrary string using ethers' signMessage (EIP-191 `personal_sign` prefix).

```
export async function signMessage(identity: HDNodeWallet, message: string): Promise<string> {
  return await identity.signMessage(message);
}
```

- **Input:** identity — an HDNodeWallet from `getIdentityFromMnemonic`; message — the raw text payload.
- **Return:** 0x-prefixed hex signature string (65 bytes: r || s || v).

11.2.4 `authenticate(mnemonicPhrase: string): Promise<{ token: string; address: string; username: string }>`

Performs the full three-phase authentication handshake with the backend.

11.2.4.1 Phase 1 — Request Nonce

POST {PUBLIC_BACKEND_URL}/auth/request-nonce?address=<checksummedAddress>

- **Query param:** address — the checksummed Ethereum address from HDNodeWallet.address.
- **Response (200):** { nonce: string } — a UUID v4 challenge.
- **Errors:** 429 (rate-limited) — reads error.error field; other — reads error.detail.
- **No auth required** (rate-limited by source IP / address).

11.2.4.2 Phase 2 — Local Signing

The message string signed (personal_sign via sign-Message):

Sign in to Playlink\nNonce: <nonce>

The literal newline is a single \n (U+000A) character. No trailing newline. This is the exact payload passed to `identity.signMessage()`.

11.2.4.3 Phase 3 — Verify

POST {PUBLIC_BACKEND_URL}/auth/verify

Content-Type: application/json

```
{
  "address": "<checksummedAddress>",
  "nonce": "<nonce>",
  "signature": "<0x-signed-message>"
}
```

- **Response (200):** { token: string; username: string } — the JWT is also set as an httpOnly session cookie by the backend via Set-Cookie. The frontend extracts only username from the response body; token is informational.
- **Errors:** Reads error.detail from response body.

11.2.4.4 Return value

```
{ token: string; address: string; username: string }
```

Side effect: The private key is saved to sessionStorage via `saveSigningKey(identity.privateKey)` immediately after derivation (see `signingKey.ts`).

11.2.4.5 Consumed by [Routes](#) that require an authenticated user (login page, room-creation forms).

11.3 signing.ts — Chat Message Signing Contract

Per-message EIP-191 signing so the backend can cryptographically prove authorship (issue #59). The canonical payload string **MUST** match the backend's `_chat_signing_message` byte-for-byte (see [../backend/realtime.md](https://github.com/ethers-io/ethers.js/blob/master/docs/advanced/realtime.md)).

11.3.1 ChatSigner

```
export interface ChatSigner {  
  signMessage(message: string): Promise<string>;  
}
```

Any object with a `signMessage` method returning a 0x-hex EIP-191 signature — `ethers.HDNodeWallet` and `ethers.Wallet` both satisfy this interface.

11.3.2 buildChatSigningMessage(room: string, content: string, sentAt: string): string

Constructs the canonical plaintext that is signed for a chat message.

```
export function buildChatSigningMessage(room: string, content: string, sentAt: string): string  
  return `PlayLink signed chat message\nroom=${room}\nsent_at=${sentAt}\ncontent=${content}`;  
}
```

Format (verbatim, with literal `\n` (U+000A) line breaks):

```
PlayLink signed chat message  
room=<room>  
sent_at=<sentAt>  
content=<content>
```

- `room`: the room name (already `encodeURIComponent`-encoded by the URL — here it is the plain room name the user sees).
- `sentAt`: ISO 8601 string from `new Date().toISOString()`.
- `content`: the trimmed message body.

11.3.3 signChatMessage(signer: ChatSigner, room: string, content: string, sentAt: string): Promise<string>

Builds the canonical message via `buildChatSigningMessage` and signs it.

```
export async function signChatMessage(  
  signer: ChatSigner,  
  room: string,  
  content: string,  
  sentAt: string  
) : Promise<string> {  
  return signer.signMessage(buildChatSigningMessage(room, content, sentAt));  
}
```

- **Return:** 0x-prefixed hex EIP-191 signature.

11.4 signingKey.ts — Session Key Custody

Stores the BIP39-derived ECDSA private key in `sessionStorage` for the duration of the browser tab session. The key is **never persisted to disk, cookie, or localStorage** — it lives only in

session-scoped memory.

Security model: The private key is XSS-exposed (the same exposure class as any in-page secret in a self-custody web app without a browser extension).

```
const STORAGE_KEY = 'playlink.sk';
```

11.4.1 saveSigningKey(privateKey: string): void

Persists the signing key for the current tab session. No-op when not running in the browser (\$app/environment browser guard).

```
export function saveSigningKey(privateKey: string): void {
  if (!browser) return;
  try {
    sessionStorage.setItem(STORAGE_KEY, privateKey);
  } catch (e) {
    console.error('Could not persist signing key', e);
  }
}
```

11.4.2 loadSigner(): Wallet | null

Returns a signer for the stored key, or null when none is available or the stored key is invalid. Invalid keys are silently removed.

```
export function loadSigner(): Wallet | null {
  if (!browser) return null;
  const privateKey = sessionStorage.getItem(STORAGE_KEY);
  if (!privateKey) return null;
  try {
    return new Wallet(privateKey);
  } catch (e) {
    console.error('Stored signing key is invalid; discarding', e);
    sessionStorage.removeItem(STORAGE_KEY);
    return null;
  }
}
```

- **Return:** ethers.Wallet (satisfies ChatSigner) or null.

11.4.3 clearSigningKey(): void

Forgets the signing key (called on logout). No-op when not in browser.

```
export function clearSigningKey(): void {
  if (!browser) return;
  sessionStorage.removeItem(STORAGE_KEY);
}
```

11.5 roomsStore.ts — Live Room List Store

WebSocket-based store that maintains the list of active game rooms. Connects to \${PUBLIC_WS_URL}/ws/rooms.

11.5.1 RoomSummary

```
export interface RoomSummary {
  name: string;
  game: string;
  lobby_location: string;
  players_active: number;
  players_max: number;
  member_addresses: string[];
  description: string | null;
  communicator_link: string | null;
  requirements: string | null;
  expires_at: string;
}
```

11.5.2 isNullableString(value: unknown): value is string | null

Type guard helper for nullable string fields.

```
export function isNullableString(value: unknown): value is string | null {
  return value === null || typeof value === 'string';
}
```

11.5.3 isRoomSummary(value: unknown): value is RoomSummary

Validates an untrusted value against the RoomSummary shape. Every frame from the rooms WebSocket is validated through this guard.

```
export function isRoomSummary(value: unknown): value is RoomSummary {
  if (typeof value !== 'object' || value === null) return false;
  const room = value as Record<string, unknown>;
  return (
    typeof room.name === 'string' &&
    typeof room.game === 'string' &&
    typeof room.lobby_location === 'string' &&
    typeof room.players_active === 'number' &&
    typeof room.players_max === 'number' &&
    Array.isArray(room.member_addresses) &&
    isNullableString(room.description) &&
    isNullableString(room.communicator_link) &&
    isNullableString(room.requirements) &&
    typeof room.expires_at === 'string'
  );
}
```

11.5.4 roomsStore: Readable<RoomSummary[]> & { destroy(): void }

A singleton store exported from the module. Created via a private createRoomsStore() factory.

```
export const roomsStore = createRoomsStore();
```

WebSocket connection:

- **URL:** \${PUBLIC_WS_URL}/ws/rooms
- **Protocol:** Plain WebSocket (no authentication — room list is public).

- **Frame format:** Bare JSON array of RoomSummary objects. There is no wrapping { type: ... } envelope; the raw array is validated with `Array.isArray(data) && data.every(isRoomSummary)`.
- **On message:** Parses JSON, validates as RoomSummary[], then `set(data)` on the underlying writable.
- **Reconnect:** Exponential backoff with +50% random jitter:
 - Base delay: 1 000 ms
 - Max delay: 30 000 ms
 - Formula: `delay = min(30_000, 1_000 * 2^attempts) + random * (0.5 * exponential)`.

Reconnect resets to 0 after a successful open.

- **destroy():** Tears down the socket, clears pending reconnect timeouts, sets `isTornDown = true` to suppress future reconnects.

Consumed by: [Route components](#) showing the lobby/room list.

11.6 chatStore.ts — Per-Room Chat WebSocket Store

Factory-based store for a single room's chat. Each room gets its own `createChatStore()` instance.

11.6.1 Public Types

11.6.1.1 ChatMessage

```
export interface ChatMessage {
  id: number;
  sender_address: string;
  sender_username: string;
  content: string;
  created_at: string;
  kind?: 'user' | 'system';
  /** EIP-191 signature, or null for legacy/unsigned. */
  signature?: string | null;
  /** Exact client timestamp that was signed, or null for legacy/unsigned. */
  sent_at?: string | null;
  /** True when the server recovered a valid signer. */
  verified?: boolean;
}
```

11.6.1.2 RoomMember

```
export interface RoomMember {
  address: string;
  username: string;
  is_admin: boolean;
}
```

11.6.1.3 RsvpStatus

```
export type RsvpStatus = 'present' | 'absent' | 'maybe';
```

11.6.1.4 RsvpEntry

```
export interface RsvpEntry {  
  address: string;  
  username: string;  
  status: RsvpStatus;  
  updated_at: string;  
}
```

11.6.1.5 RoomEventState

```
export interface RoomEventState {  
  starts_at: string;  
  ends_at: string;  
  created_by: string;  
  created_at: string;  
  updated_at: string;  
  rsvps: RsvpEntry[];  
}
```

11.6.2 Validators

Each validator function checks the structural type at runtime, matching the exact field shapes above.

Function	Signature
isChatMessage	(value: unknown) => value is ChatMessage
isRoomMember	(value: unknown) => value is RoomMember
isRsvpStatus	(value: unknown) => value is RsvpStatus
isRsvpEntry	(value: unknown) => value is RsvpEntry
isRoomEventState	(value: unknown) => value is RoomEventState

11.6.3 parseFrame(raw: string): ChatFrame | null

Parses and validates a raw WebSocket message string. Returns a discriminated ChatFrame object on success, or null when the JSON is malformed or the frame type is unknown.

Frame types handled (see [../backend/realtime.md](https://github.com/andrewcsc/realtime/blob/master/backend/realtime.md) for the server-side spec):

type	Extra Fields	Store Action
history	messages: ChatMessage[]	messages.set(messages) — replaces all
message	message: ChatMessage	messages.update(msgs => [...msgs, msg])
event_update	event: RoomEventState \ null	event.set(event)
rsvp_update	rsvp: RsvpEntry	event.update() — merges RSVP, dedicated by case-insensitive address
roster_update	members: RoomMember[]	members.set(members) — replaces all

type	Extra Fields	Store Action
room_closed	room: string	closed.set(true); isTornDown = true; socket closed
member_kicked	member_address, member_username, created_by, ownership_transferred: boolean	owner.set(created_by); if member_address matches options.currentAddress, kicked.set(true) and isTornDown = true
error	detail: string	console.warn('Chat message rejected:', detail)

11.6.4 ChatStore

```
export interface ChatStore {
  messages: Readable<ChatMessage[]>;
  event: Readable<RoomEventState | null>;
  members: Readable<RoomMember[]>;
  closed: Readable<boolean>;
  owner: Readable<string>;
  kicked: Readable<boolean>;
  send(content: string): Promise<void>;
  destroy(): void;
}
```

Readable	Initial Value	Description
messages	[]	Live chat history for the room.
event	options.initialEvent ?? null	Current scheduled event (or null when none).
members	options.initialMembers ?? []	Live room roster, updated on join/leave.
closed	false	Becomes true when an admin closes the room (room_closed).
owner	options.initialOwner ?? ''	Current room owner address. Updated after a creator kick.
kicked	false	Becomes true when this client is removed from the room.

11.6.5 CreateChatStoreOptions

```
export interface CreateChatStoreOptions {
  initialEvent?: RoomEventState | null;
  initialMembers?: RoomMember[];
  initialOwner?: string;
  currentAddress?: string;
  signer?: ChatSigner | null;
}
```

Option	Purpose
<code>initialEvent</code>	Pre-populated event state from SSR, avoids flash of empty content.
<code>initialMembers</code>	Pre-populated roster from SSR, used until first WS update.
<code>initialOwner</code>	Room owner address from SSR.
<code>currentAddress</code>	Address of this browser session. Used for kick self-detection.
<code>signer</code>	Key for signing outgoing messages (issue #59). When absent, messages sent unsigned.

11.6.6 `createChatStore(roomName: string, token: string, options?: CreateChatStoreOptions): ChatStore`

Factory that returns a `ChatStore` for a specific room.

WebSocket connection:

- **URL:** `${PUBLIC_WS_URL}/ws/rooms/${encodeURIComponent(roomName)}/chat?token=${encodeURIComponent(token)}`
- **Auth:** JWT passed as query parameter (browsers cannot set custom WebSocket headers).
- **Reconnect:** Same exponential backoff as `roomsStore` (base 1 000 ms, max 30 000 ms, +50% jitter). Reconnect is suppressed after closed, kicked, or close codes 4403/4409.

Send behavior — `async send(content: string): Promise<void>`:

1. Trims content. No-ops if empty or socket not OPEN.
2. If `options.signer` is set, generates `sentAt = new Date().toISOString()`, calls `signChatMessage(signer, roomName, trimmed, sentAt)`, and sends `{ "content": "<trimmed>", "sent_at": "<sentAt>", "signature": "<0x-hex>" }`.
3. On signing failure, falls back to unsigned `{ "content": "<trimmed>" }` with `console.error`.
4. If no signer, sends unsigned `{ "content": "<trimmed>" }`.

Close code handling:

Close Code	Behavior
4403	Not a member of this room — <code>kicked.set(true)</code> , no reconnect.
4409	Unknown room — <code>kicked.set(true)</code> , no reconnect.
Others	Schedule reconnect (unless torn down).

11.6.7 Lifecycle

```
const store = createChatStore(roomName, token, { signer, currentAddress });
// ... use store.messages, store.send(...) ...
store.destroy(); // tear down socket, cancel reconnect
```

Consumed by: [Route components](#) for the per-room chat page. See also [../backend/realtime.md](#) for the WebSocket frame protocol from the server side.

11.7 `hintsContext.svelte.ts` — Svelte 5 Runes Hints System

Reactive context for populating contextual hint/tooltip overlays. Uses Svelte 5 `$state` runes.

11.7.1 HintTone

```
export type HintTone = 'gold' | 'green' | 'amber' | 'red' | 'stone' | 'blue';
```

11.7.2 HintEntry

```
export interface HintEntry {  
  key: string;  
  label: string;  
  tone?: HintTone;  
  onclick?: () => void;  
}
```

11.7.3 HintsState

```
export class HintsState {  
  hints = $state<HintEntry[]>([]);  
  
  constructor(initial: HintEntry[] = []) {  
    this.hints = initial;  
  }  
  
  set(entries: HintEntry[]): void {  
    this.hints = entries;  
  }  
  
  clear(): void {  
    this.hints = [];  
  }  
}
```

- `hints` is a reactive `$state` field — any Svelte component reading it via `getHintsState()` will re-render when it changes.

11.7.4 provideHints(initial?: HintEntry[]): HintsState

Creates a `HintsState` and sets it in the Svelte component context under a `Symbol('hints')` key. Should be called once in a layout or page component (during `<script>` initialization).

```
export function provideHints(initial: HintEntry[] = []): HintsState {  
  const state = new HintsState(initial);  
  setContext(KEY, state);  
  return state;  
}
```

- **Returns:** The `HintsState` instance for direct manipulation.

11.7.5 getHintsState(): HintsState | undefined

Retrieves the `HintsState` from context. Undefined when called outside of a subtree that called `provideHints`.

```
export function getHintsState(): HintsState | undefined {  
  return getContext<HintsState | undefined>(KEY);  
}
```

11.8 lobbyLocations.ts — Lobby Location Data and Geo Helpers

11.8.1 LobbyLocation

```
export interface LobbyLocation {  
  code: string;  
  label: string;  
  lat: number;  
  lon: number;  
}
```

11.8.2 FALLBACK_LOBBY_LOCATIONS

A constant array of 10 pre-defined lobby locations covering major regions:

Code	Label	Lat	Lon
na-east	North America East	39.0	-77.0
na-west	North America West	37.8	-122.4
eu-west	Europe West	51.5	-0.1
eu-central	Europe Central	50.1	8.7
eu-north	Europe North	59.3	18.1
sa-east	South America East	-23.6	-46.6
asia-east	Asia East	35.7	139.7
asia-south	Asia South	1.3	103.8
oceania	Oceania	-33.9	151.2
africa-south	Africa South	-26.2	28.0

11.8.3 FALLBACK_LOBBY_LOCATION

```
export const FALLBACK_LOBBY_LOCATION = 'eu-central';
```

The fallback lobby location code used when no location is set.

11.8.4 isLobbyLocation(value: unknown): value is LobbyLocation

Runtime type guard for LobbyLocation.

```
export function isLobbyLocation(value: unknown): value is LobbyLocation {  
  if (typeof value !== 'object' || value === null) return false;  
  const location = value as Record<string, unknown>;  
  return (  
    typeof location.code === 'string' &&  
    typeof location.label === 'string' &&  
    typeof location.lat === 'number' &&  
    typeof location.lon === 'number'  
  );  
}
```

11.8.5 lobbyLocationLabel(locations: LobbyLocation[], code: string): string

Resolves a location code to its human-readable label.

```
export function lobbyLocationLabel(locations: LobbyLocation[], code: string): string {  
  return locations.find((location) => location.code === code)?.label ?? code;  
}
```

- **Return:** The matching location's label, or code itself when not found.

11.8.6 distanceKm(a, b): number

Computes the great-circle distance between two geographic coordinates using the Haversine formula.

```
export function distanceKm(
  a: Pick<LobbyLocation, 'lat' | 'lon'>,
  b: Pick<LobbyLocation, 'lat' | 'lon'>
): number {
  const toRad = (deg: number) => (deg * Math.PI) / 180;
  const earthRadiusKm = 6371;
  const dLat = toRad(b.lat - a.lat);
  const dLon = toRad(b.lon - a.lon);
  const lat1 = toRad(a.lat);
  const lat2 = toRad(b.lat);
  const h = Math.sin(dLat / 2) ** 2
    + Math.cos(lat1) * Math.cos(lat2) * Math.sin(dLon / 2) ** 2;
  return 2 * earthRadiusKm * Math.asin(Math.sqrt(h));
}
```

- **Input:** Two objects with lat/lon properties (satisfies Pick<LobbyLocation, 'lat' | 'lon'>).
- **Return:** Distance in kilometres.

11.8.7 nearestLobbyLocation(locations: LobbyLocation[], coords): LobbyLocation | null

Finds the nearest lobby location to a given coordinate pair by iterating all locations and tracking minimum distance.

```
export function nearestLobbyLocation(
  locations: LobbyLocation[],
  coords: Pick<LobbyLocation, 'lat' | 'lon'>
): LobbyLocation | null {
  let nearest: LobbyLocation | null = null;
  let nearestDistance = Number.POSITIVE_INFINITY;
  for (const location of locations) {
    const d = distanceKm(coords, location);
    if (d < nearestDistance) {
      nearestDistance = d;
      nearest = location;
    }
  }
  return nearest;
}
```

- **Return:** The LobbyLocation with the smallest Haversine distance, or null when the array is empty.

11.9 hooks.server.ts — Session Hook

The SvelteKit server handle hook that runs on every server-side request. Reads the session httpOnly cookie, decodes the JWT, validates expiration, and populates event.locals.user.

```
import { jwtDecode } from 'jwt-decode';
import type { Handle } from '@sveltejs/kit';

interface SessionTokenClaims {
  sub?: string;
  exp?: number;
  is_admin?: boolean;
}

export const handle: Handle = async ({ event, resolve }) => {
  const session = event.cookies.get('session');

  if (session) {
    try {
      const decoded = jwtDecode<SessionTokenClaims>(session);

      if (decoded.exp && decoded.exp * 1000 < Date.now()) {
        throw new Error('Token expired');
      }

      if (decoded.sub) {
        event.locals.user = {
          address: decoded.sub,
          isAdmin: decoded.is_admin === true
        };
      }
    } catch {
      event.cookies.delete('session', { path: '/' });
    }
  }

  return await resolve(event);
};
```

11.9.1 Behavior

- **No session cookie:** event.locals.user remains undefined. The request proceeds without authentication.
- **Valid JWT:** Decodes the token, checks exp (in seconds — converted to ms for comparison). Sets event.locals.user = { address: decoded.sub, isAdmin: decoded.is_admin === true }.
- **Expired or malformed JWT:** Deletes the session cookie (with path: '/'), leaving event.locals.user undefined.

11.10 app.d.ts — Global Type Augmentation

```
declare global {
  namespace App {
```

```

    interface Locals {
      user?: {
        address: string;
        isAdmin: boolean;
      };
    }
  }
}

export {};

```

Augments SvelteKit's `App.Locals` so that `event.locals.user` is type-safe in hooks, load functions, and form actions across the application. `user` is optional (undefined when no valid session cookie exists); when present, `address` is always a string and `isAdmin` is always boolean.

11.11 Cross-References

- [Route components](#) — consumers of all stores and auth functions.
- [Backend realtime](#) — WebSocket frame protocol, close codes, and the `_chat_signing_message` builder.
- [Backend API reference](#) — REST endpoints for auth.
- [Data model](#) — room and event schema.
- [Architecture](#) — system-wide data flow.

12 Deployment & Operations

Reference for local development, Docker infrastructure, container startup, and CI/CD pipeline for the Playlink project.

Source: `docker-compose.yml`, `backend/Dockerfile`, `backend/entrypoint.sh`, `frontend/Dockerfile`, `.github/workflows/cicd.yml`, `.env.example`, `frontend/.env.example`, `backend/database.py`

12.1 Local Development Quick Start

```

# 1. Copy environment files
cp .env.example .env
cp frontend/.env.example frontend/.env

```

```

# 2. Build and launch all services
docker compose up --build

```

After startup:

Service	URL	Notes
Frontend (SvelteKit)	<code>http://localhost:3000</code>	Dev server via <code>@sveltejs/adaptor-node</code>
Backend (FastAPI)	<code>http://localhost:8000</code>	API root; OpenAPI docs at <code>/docs</code>
Postgres 18	<code>db:5432 (internal)</code>	No host port mapping — see rationale below

12.1.1 Database port visibility

The db service in `docker-compose.yml` deliberately omits a `ports:` directive. Docker’s iptables rules bypass UFW and similar host firewalls, so even a guarded `5432:5432` would expose Postgres to the public internet. The database is reachable only on the internal bridge network (`app-network`).

12.2 Docker Compose Breakdown

12.2.1 Network: `app-network`

Single bridge network shared by all three services. No external connectivity required — inter-service communication uses service names as hostnames (e.g. `http://backend:8000`).

12.2.2 Volume: `postgres_data`

Named Docker volume mounted at `/var/lib/postgresql` inside the db container. Survives container restarts; delete manually to reset the database.

12.2.3 db (Postgres 18)

Field	Value
Image	<code>postgres:18-alpine</code>
Env file	<code>.env (root)</code>
Ports	None — see security rationale above
Volume	<code>postgres_data:/var/lib/postgresql</code>
Network	<code>app-network</code>
Healthcheck	<code>pg_isready -U \$POSTGRES_USER -d \$POSTGRES_DB — interval 10s, timeout 5s, retries 5</code>

12.2.4 backend (FastAPI)

Field	Value
Build	<code>./backend/Dockerfile</code>
Ports	<code>8000:8000</code>
Env file	<code>.env (root)</code>
Depends on	<code>db condition: service_healthy</code>
Network	<code>app-network</code>
Healthcheck	<code>Python urllib GET http://localhost:8000/ — interval 10s, timeout 5s, retries 5, start_period 5s</code>

12.2.5 frontend (SvelteKit)

Field	Value
Build	<code>./frontend/Dockerfile</code>
Ports	<code>3000:3000</code>

Field	Value
Env file	frontend/.env
Environment	BACKEND_INTERNAL_URL=http://backend:8000, ORIGIN (default http://localhost:3000)
Depends on	backend condition: service_healthy
Network	app-network

BACKEND_INTERNAL_URL — used by SvelteKit server-side load functions and form actions for in-cluster calls to the FastAPI backend. Browser-side code uses **PUBLIC_BACKEND_URL** / **PUBLIC_WS_URL** from the frontend .env.

ORIGIN — required by @sveltejs/adaptor-node for its CSRF check. The value must match the URL the browser navigates to. Defaults to http://localhost:3000 for local development; override in the production .env.

12.3 Container Startup

12.3.1 Backend multi-stage Dockerfile

FROM ghcr.io/astral-sh/uv:python3.14-bookworm-slim

Two-stage uv sync:

1. **Dependencies only** — mounts uv.lock and pyproject.toml from build context, runs uv sync --frozen --no-install-project --no-dev. Layers cached so source edits don't re-resolve.
2. **Project install** — copies full source, runs uv sync --frozen --no-dev.

Environment: UV_COMPILE_BYTECODE=1, UV_LINK_MODE=copy, PATH includes /app/.venv/bin.

The ENTRYPOINT is reset to empty (the base image's uv entrypoint is discarded). Non-root user nonroot (uid/gid 999) is created at build time.

CMD ["/app/entrypoint.sh"]

12.3.2 Backend entrypoint (entrypoint.sh)

```
#!/bin/sh
set -e

for i in $(seq 1 30); do
    python - <<'PY' && break
import socket, sys
try:
    socket.getaddrinfo("db", 5432)
    s = socket.create_connection(("db", 5432), timeout=2)
    s.close()
except Exception:
    sys.exit(1)
PY
    echo "DB not ready yet ($i/30)"
    sleep 2
done
```

```
alembic upgrade head
exec uvicorn main:app --host 0.0.0.0 --port 8000
```

Steps, in order:

1. **DB readiness loop** — up to 30 attempts (2 s apart) testing TCP connectivity to host db port 5432 via raw Python socket. Exits the loop early on success, fails hard after exhausting retries.
2. **alembic upgrade head** — applies all pending migrations to the connected database.
3. **exec uvicorn main:app ...** — replaces the shell process with the ASGI server.

12.3.3 Frontend multi-stage Dockerfile

FROM oven/bun:1-slim AS base

Three stages:

Stage	Purpose
base	Sets WORKDIR /usr/src/app
build	Installs deps (bun install --frozen-lockfile), copies source, runs bun run build
runtime	Copies production deps and build/ output, sets NODE_ENV=production, HOST=0.0.0.0, PORT=3000

Runtime command:

```
CMD ["bun", "run", "build/index.js"]
```

No .env is baked into the image — runtime env vars are injected via docker compose env_file / environment.

12.4 Docker-Aware DATABASE_URL

In backend/database.py (and replicated in alembic/env.py):

```
if "@db:" in DATABASE_URL and not Path("/.dockerenv").exists():
    DATABASE_URL = DATABASE_URL.replace("@db:", "@localhost:")
```

When the backend detects it is NOT running inside a Docker container (no /.dockerenv file), it rewrites the @db: host component to @localhost:. This allows the same .env file to work for both Docker Compose (where db resolves via the bridge network) and local development (where Postgres runs on the host). See [configuration](#) for the full env var reference.

12.5 CI/CD Pipeline

The single workflow .github/workflows/cicd.yml runs on push to main and pull requests targeting main.

12.5.1 Jobs

Job	Runner	Dependencies	Trigger Condition	Key Steps
backend-lint	ubuntu-latest	—	push/PR to main	uv setup, ruff check ., ruff format --check
backend-test	ubuntu-latest	—	push/PR to main	uv setup, pytest --cov=... --cov-fail-under=80 with DATA-BASE_URL=sqlite:/// and JWT_SECRET=ci-test-secret-at-least-thirty-two-chars
backend-migration	ubuntu-latest	Postgres service container	push/PR to main	uv setup, alembic upgrade head, alembic downgrade -1 && alembic upgrade head (round-trip), alembic check (drift detection)
frontend-lint	ubuntu-latest	—	push/PR to main	bun install --frozen-lockfile, bun run lint, bun x prettier --check .
frontend-test	ubuntu-latest	—	push/PR to main	bun install --frozen-lockfile, bun run test --coverage
build-test	ubuntu-latest	—	push/PR to main	docker compose build (verifies images compile)
deploy	ubuntu-latest	All 5 preceding jobs	push to main ONLY	Tailscale connect, SSH into production, write .env from secrets/vars, git pull, docker compose up --build -d

12.5.2 Deployment details (auto-deploy on merge to main)

The deploy job runs only when all preceding jobs pass AND the trigger is a push event on refs/heads/main.

- Establishes a Tailscale connection using `{{ secrets.TS_AUTHKEY }}`.
- Connects to the production server via SSH (`appleboy/ssh-action@v1`) authenticated with `{{ secrets.SERVER_SSH_KEY }}`.
- Writes a fresh `.env` to `~/playlink/.env` using GitHub Actions secrets (sensitive values) and vars (non-sensitive defaults), plus a separate `frontend/.env` with `PUBLIC_WS_URL` and `PUBLIC_BACKEND_URL`.
- Runs `git pull origin main`, then `docker compose up --build -d` with `DOCKER_BUILDKIT=1`.

Typical deploy duration: 30 s - 5 min depending on image cache state and network latency.

12.5.3 Environment variables injected at deploy time

Root .env (production): - `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`, `POSTGRES_HOST=db`, `POSTGRES_PORT=5432` - `DATABASE_URL=postgresql+psycopg://<user>:<password>@db:5432/<db>`
- `JWT_SECRET`, `JWT_ALGORITHM` - `NONCE_EXPIRATION_MINUTES`, `JWT_EXPIRATION_MINUTES` - `ORIGIN=https://playlink.bartek.monster` - `ADMIN_ADDRESSES`

Frontend .env (production): - `PUBLIC_WS_URL=wss://<backend-domain>` - `PUBLIC_BACKEND_URL=https://<backend-domain>`

12.6 Related Documentation

- [Configuration](#) — all backend env var definitions and validation
- [Testing](#) — local test suite reference
- [Migrations](#) — Alembic chain and migration workflow
- [Architecture](#) — system context and component relationships
- [Index](#) — full documentation map

13 Testing Reference

This document describes how to run and interpret the test suites for both the backend (Python/FastAPI) and frontend (SvelteKit/TypeScript) components of Playlink.

Source: `backend/tests/conftest.py`, `backend/tests/test_*.py`, `backend/pyproject.toml`, `frontend/vite.config.js`, `frontend/package.json`, `frontend/src/test/*.test.ts`, `.github/workflows/cicd.yml`

13.1 Backend Tests

The backend test suite uses **pytest** with coverage via **pytest-cov**. All tests target an ephemeral in-memory SQLite database so no Postgres instance is required for unit testing.

13.1.1 Running

```
cd backend
uv run pytest
```

Arguments are forwarded normally:

```

uv run pytest -v                # verbose output
uv run pytest -k test_auth      # run only auth-related tests
uv run pytest --coverage        # (alias for --cov=...; see CI command below)

```

The CI pipeline runs the full coverage gate:

```

uv run pytest \
  --cov=main --cov=models --cov=database --cov=usersnames \
  --cov-report=term-missing:skip-covered \
  --cov-fail-under=80

```

This enforces at least **80% line coverage** across the four tracked modules (main, models, database, usernames). Skipped-covered lines are hidden in the terminal report.

13.1.2 Configuration

From backend/pyproject.toml:

```

[tool.pytest.ini_options]
pythonpath = ["."]
testpaths = ["tests"]
env = [
    "DATABASE_URL=sqlite://",
    "JWT_SECRET=this-is-a-very-secure-test-secret-at-least-32-chars",
]

```

The environment is set automatically by `pytest -env` — tests always see `DATABASE_URL=sqlite://` and a stable `JWT_SECRET`.

13.1.3 Fixtures (conftest.py)

All fixtures live in backend/tests/conftest.py:

Fixture	Scope	Purpose
session (name="session")	function	Creates in-memory SQLite engine (sqlite://) with StaticPool and check_same_thread=False. Creates all tables, seeds the game table with 5 reference titles (Quake III Arena, Diablo II, StarCraft, Half-Life, Unreal Tournament). Yields a Session. Disposes engine in finally.
client (name="client")	function	Overrides the get_session FastAPI dependency with the test session. Yields a fastapi.testclient.TestClient. Clears dependency_overrides in teardown.
disable_rate_limit (autouse)	function	Sets app.state.limiter.enabled = False so all endpoints are callable without hitting the rate limiter.

The session fixture uses `sqlmodel.pool.StaticPool` to avoid SQLite threading issues and reuses the same connection for the entire test function.

13.1.4 Coverage Areas

Test file	Coverage
<code>test_auth.py</code> (5 tests)	Happy-path authentication flow: request nonce, verify EIP-191 signature, GET <code>/users/me</code> , nonce invalidation chain.
<code>test_auth_edges.py</code> (8 tests)	Edge cases: nonce hashing & DB shape, replay attack, expired nonce, bad signature format, expired/missing-sub/not-found-user JWT, forged admin claim.
<code>test_rooms.py</code> (16 tests)	Room join/leave lifecycle: auth required, 404 on missing room, duplicate name (409), 3-room per-user limit, full room rejection, non-member leave, member payload shape updates.
<code>test_rooms_lifecycle.py</code> (9 tests)	Room CRUD with metadata: lobby_location validation, oversized description (422), unknown location (400), custom game auto-add, <code>/lobby-locations</code> endpoint.
<code>test_room_events.py</code> (25+ tests)	Event scheduling (PUT/GET/DELETE), RSVP upsert & idempotency, reschedule clears RSVPs, leave clears RSVP, expiry pruning, WebSocket broadcasts (<code>event_update</code> , <code>rsvp_update</code> , <code>roster_update</code>), payload shape parity.
<code>test_chat.py</code> (10+ tests)	Chat WebSocket: bad token/not-member/missing-room disconnect, member broadcast, history replay, oversize message drop, EIP-191 signed message verification (valid, tampered, wrong signer, stale timestamp, unsigned, history includes signature).
<code>test_kick.py</code> (8 tests)	Admin kick: auth required, cannot kick admin, RSVP removal + rejoin allowed, creator transfer, WS disconnect (4409) + notifications, admin exempt from 3-room limit, member payload <code>is_admin</code> flag.
<code>test_admin.py</code> (12 tests)	Admin auth enforcement (401/403), delete room cascade (messages, events, RSVPs, members) + WebSocket <code>room_closed</code> broadcast, game CRUD with <code>force</code> flag.
<code>test_admin_edges.py</code> (3 tests)	Admin edge cases: trimmed game name, force-delete game WS notifications across multiple rooms, broad WS connection handling.

Test file	Coverage
test_users.py (11 tests)	Username validation unit tests (stoplist, profanity, format codes), PATCH /users/me (success, idempotent, bad format, profane, duplicate conflict, unauthenticated).
test_cleanup.py (2 tests)	Expired room pruning via get_rooms_payload: removes expired rooms + messages, preserves valid rooms.
test_helpers.py (8 tests)	Unit tests for module-level helpers: _parse_admin_addresses, is_admin_address case-insensitivity, hash_nonce stability, _iso datetime formatting, _members_payload shape, _msg_dict shape, ConnectionManager/RoomChatManager stale socket cleanup.
test_migrations.py (1 test)	Alembic smoke test: applies all migrations to empty SQLite in a temp directory, asserts all 8 tables exist, asserts room columns (description, communicator_link, requirements) and roomevent columns (starts_at, ends_at, created_by, updated_at).

13.1.5 WebSocket Coverage

The TestClient supports WebSocket calls via `client.websocket_connect("/ws/rooms/{room_name}/chat?token={token}")`. Tests in `test_chat.py`, `test_room_events.py`, `test_kick.py`, `test_admin.py`, and `test_admin_edges.py` exercise real WS frames against the running app instance within the test session.

13.1.6 Alembic Migration Tests

`test_migrations.py` runs `alembic upgrade head` against a fresh SQLite database in a `tmp_path`, then inspects the resulting schema for all expected tables and columns. A separate CI job (backend-migrations) applies migrations to a real Postgres container and additionally runs `alembic downgrade -1 + alembic upgrade head` round-trip and `alembic check` for model/migration drift detection.

13.2 Frontend Tests

The frontend test suite uses **Vitest** with **jsdom** environment. All tests are unit/integration — no browser is launched.

13.2.1 Running

```
cd frontend
bun run test
```

With coverage:

```
bun run test -- --coverage
```

13.2.2 Configuration

From frontend/vite.config.js:

```
test: {
  globals: true,
  environment: 'jsdom',
  setupFiles: ['./src/test/setup.ts']
}
```

- `globals: true` — `describe`, `it`, `expect`, `vi` are available without explicit imports (`vitest/globals` in `tsconfig.json` adds type support).
- `environment: 'jsdom'` — provides a browser-like DOM (`document`, `window`, `sessionStorage`, `FormData`, etc.) without a real browser.
- `setupFiles: ['./src/test/setup.ts']` — imports `@testing-library/jest-dom` for DOM matchers (`toBeInTheDocument`, etc.).

The `tsconfig.json` includes `"types": ["vitest/globals"]` for type-aware autocompletion.

13.2.3 Coverage Areas

Test file	Coverage
<code>signing.test.ts</code>	Validates canonical chat message format from <code>signing.ts</code> — special characters preserved, <code>buildChatSigningMessage</code> produces exact payload, <code>signChatMessage</code> delegates to signer with correct message.
<code>signingKey.test.ts</code>	<code>saveSigningKey</code> , <code>loadSigner</code> , <code>clearSigningKey</code> — verifies <code>sessionStorage</code> round-trip and invalid key removal.
<code>roomsStore.test.ts</code>	Type guards <code>isNullableString</code> and <code>isRoomSummary</code> from <code>roomsStore.ts</code> — edge cases for missing fields, wrong types, nullable fields.
<code>chatStore.test.ts</code>	Validators and <code>parseFrame</code> from <code>chatStore.ts</code> — <code>isChatMessage</code> , <code>isRoomMember</code> , <code>isRsvpStatus</code> , <code>isRsvpEntry</code> , <code>isRoomEventState</code> , frame parsing for message/unknown/bad JSON.
<code>lobbyLocations.test.ts</code>	<code>isLobbyLocation</code> , <code>lobbyLocationLabel</code> , <code>distanceKm</code> (Haversine: zero distance, London→Frankfurt ~640 km), <code>nearestLobbyLocation</code> , <code>FALLBACK_LOBBY_LOCATION</code> exists in list.
<code>hintsContext.svelte.test.ts</code>	<code>HintsState</code> class — initialization with/without hints, <code>set()</code> , <code>clear()</code> methods. Svelte 5 runes (<code>\$state</code>) integration.
<code>auth.page.server.test.ts</code>	<code>+page.server.ts</code> load and actions — no session → user: null, valid token → user decoded, invalid token cleared; login action (missing token, stores cookie); logout action (deletes cookie).

Test file	Coverage
<code>profile.page.server.test.ts</code>	+ <code>page.server.ts</code> load and update action — redirects without session, loads profile from API, redirects on invalid session; update requires auth, rejects empty username, updates via PATCH.
<code>rooms.page.server.test.ts</code>	+ <code>page.server.ts</code> load and actions — loads without login, loads logged-in user; create rejects unauthenticated/missing-fields; addGame rejects empty name.
<code>rooms.name.page.server.test.ts</code>	+ <code>page.server.ts</code> load and actions — redirects when room missing (303 → /rooms); setRsvp rejects missing auth and invalid status; kickMember rejects missing member.

13.2.4 Route-Level Tests

The four route tests (`auth.page.server.test.ts`, `profile.page.server.test.ts`, `rooms.page.server.test.ts`, `rooms.name.page.server.test.ts`) exercise SvelteKit server load functions and form actions directly by importing from the route modules. They mock `$env/dynamic/public`, `$env/dynamic/private`, `jwt-decode`, `globalThis.fetch`, and the `cookies` object to simulate server-side request context. These are not full end-to-end tests — they verify the server-side logic (auth guards, API proxying, validation) without starting the SvelteKit dev server.

13.3 Related Documentation

- [REST API Reference](#) — endpoint contracts tested by the backend suite.
- [WebSocket Protocol Reference](#) — WS frame types verified by `test_chat`, `test_room_events`, `test_kick`, and `test_admin`.
- [Deployment Guide](#) — CI pipeline integration, migration smoke tests, and coverage gates.